

Pickup & Delivery with Time Windows and Transfers: combining decomposition with metaheuristics

Ioannis Avgerinos¹, Ioannis Mourtos¹, Nikolaos Tsompanidis¹, and Georgios Zois^{1, 2}

¹ELTRUN Lab, Department of Management Science and Technology, Athens University of Economics and Business, Patision 76, 10434, Athens, Greece E-mail: iavgerinos@aueb.gr, mourtos@aueb.gr, tsompanidisnik@gmail.com, georzois@aueb.gr

²Optiscale, Athens 114 72, Greece

Abstract

This paper examines the generalisation of the Pickup and Delivery Problem that allows mid-route load exchanges among vehicles and obeys strict time-windows at all locations. We propose a novel Logic-Based Benders Decomposition (LBBD) that improves optimality gaps for all benchmarks in the literature and scales up to handle larger ones. To tackle even larger instances, we introduce a refined Large Neighborhood Search (LNS) algorithm that improves the adaptability of LNS beyond case-specific configurations appearing in related literature.

To bridge the gap in benchmark availability, we develop an instance generator that allows for extensive experimentation. For moderate datasets (25 and 50 requests), we evaluate the performance of both LBBD and LNS, the former being able to close the gap and the latter capable of providing near-optimal solutions. For larger instances (75 and 100 requests), we recreate indicative state-of-the-art metaheuristics to highlight the improvements introduced by our LNS refinements, while establishing its scalability.

Keywords. pickup and delivery problem, integer programming, logic-based benders decomposition, large neighbourhood search.

1 Background

Motivation. The widespread applicability of routing problems has driven the development of numerous variants, along with methods capable of handling large-scale instances. We investigate here a variant that enjoys increasing applicability yet remains quite challenging. Our aim is to provide a novel modelling and decomposition approach, coupled with effective exact and heuristic methods.

The traditional *Vehicle Routing Problem* (VRP) [31] assumes vehicles departing from a single depot to cover a single type of operation for all customers (i.e., requests), i.e., either pickups or deliveries. To align with industry requirements, extensions of VRP also consider multiple depots, time windows and combined pickup and delivery operations. Specifically, the *Dial-A-Ride Problem* (DARP) [8] assumes pickup and delivery locations per request and hence the precedence constraint that a vehicle must visit the pickup location of a request before its delivery location. In its original version, DARP assumes unitary loads for all requests,

as its motivation comes from passenger transport [29]. Therefore, the *Pickup and Delivery Problem* (PDP) [12] extends DARP to accommodate loads of varying size, hence forming the most generic VRP variant.

It becomes simple to see that VRP reduces to PDP by introducing one copy of the depot per request to serve as its pickup point. Also, PDP allows not just for a different depot per vehicle (as in the multi-depot variant of VRP), but for a different origin and destination depot per vehicle. What further complicates matters is that a vehicle’s load fluctuates throughout the route as pickups and deliveries intertwine. The above, combined with the precedence constraints, result in PDP being tackled for instances of minuscule size compared to the scale claimed currently for VRP [1].

Further challenges show up in urban logistics and particularly in areas having frequent and densely-concentrated requests. Routing must simultaneously handle strict time windows, restricted access for larger vehicles and rising inventory costs hence lack of warehousing facilities. This explains the appearance of innovative approaches like mobile depots [13] serving as intermediate points for load replenishment, i.e., for transfers of requests between vehicles, or crowdshipping [32] where multiple drivers coordinate to fulfill a request. Therefore, PDP naturally generalises to the *PDP with Transfers* [9] by introducing locations where loads can be exchanged between vehicles. Such transfer points impose the so called *vehicle synchronisation* constraints: a vehicle picking up a request must arrive at the transfer point after the vehicle dropping it off. Adding time windows for *all* locations (origin and destination of each vehicle, pick and delivery of each request, transfer points) yields the *Pickup and Delivery Problem with Time Windows and Transfers*, called hereafter PDPT for conciseness of our notation.

PDPT generalises the Vehicle Routing Problem with Transfers (VRPT) [11]. Although VRPT incorporates synchronisation constraints, the existing literature considers uniform demand per location rather than distinct pickup and delivery requests. This assumption enables load splitting and simplifies modelling through flow constraints, hence yielding a simpler problem. VRPT has been addressed through Column Generation [3], and a two-index MILP formulation combined with a Multi-Start Local Search [2]. The study of [33] incorporates several key aspects of the PDPT, notably multiple depots that can function as transfer points for the deployed vehicles.

The PDPT imposes several big-M constraints to a monolithic MILP, e.g., precedence constraints ensuring that each request is picked up before being delivered or time-window constraints regulating vehicle arrival times at each location. Also, the inclusion of transfer points inflates the solution space and makes it non-trivial to enforce synchronisation constraints. Furthermore, capacity constraints must be managed at every route segment, since pickups and deliveries occur throughout a vehicle’s journey. Enhancements like valid inequalities strengthening the lower bound or the elimination of some binary variables remain insufficient even for moderate instances. Current exact methods struggle to produce feasible solutions for up to 30 requests [17] even without time windows.

Therefore, we have developed a novel Logic-Based Benders Decomposition (LBBD) that reduces optimality gaps for the instances tackled by exact methods [17] and, most importantly, allows for larger and more complex instances to be tackled. For instance sizes where the efficiency of our exact method diminishes, we introduce a Large Neighbourhood Search (LNS) method that avoids instance-specific characteristics and parameter tuning. Using the LNS solution as a warm-start allows LBBD to offer lower bounds and often improve that solution.

Literature review. Mathematical models for the PDP appear in [12, 29], while comprehensive reviews

on the evolution of these models appear in [8] and in [16]. Due to the complexity of the synchronisation constraints, monolithic MILPs handle instances with very few requests, hence motivating the use of decomposition approaches. An early attempt to accelerate convergence to optimality through decomposition is the Branch-and-Cut scheme of [9]. Although the decomposition outperforms the monolithic MILP, instances with only up to 6 requests, in a specific DARP scenario involving passenger transport (i.e., with unitary loads) have been solved. The improved MILP formulation of [24] expands the solvable instance size to 14 requests. The approach of [20], combining Column Generation and Branch-and-Cut, performs well on instances with up to 193 requests but for the special PDPT case where routes of predefined links (a.k.a. shuttle routes) serve passengers for a limited number of stops.

The most recent advancement in mathematical modelling for PDPT is presented by [17] that extends [24]. This new MILP addresses instances of up to 30 requests with loose time windows but only up to 5 requests when time windows become strict. Given the absence of publicly available benchmark instances of larger size, we rely upon the work of [17] to assess our methods.

Several studies have explored the use of transshipment (i.e., transfers) in PDP and its variants by deploying various metaheuristics. Early work [21] investigates the benefits of load transfers between vehicles in courier operations and particularly in large, multi-city areas. Building on this, two nearly-concurrent studies pioneer the development of Adaptive Large Neighbourhood Search (ALNS) for the PDPT: [23] proposes a GRASP approach that employs ALNS to improve the initially constructed solutions, while [19] proposes a handful of efficient heuristics for modifying a partial solution and then incorporating it into an ALNS scheme. [10] are the first to examine the potential of Genetic Algorithms (GA), hence proposing both an LNS and a GA. Finally, transfers have been examined in crowd-sourced urban freight delivery [27], where ALNS effectively optimises transfer opportunities and vehicle synchronisation, while significantly reducing travel distances and the number of required drivers. These findings have motivated us to develop a refined LNS scheme, whose core aim is to complement the strengths of our LBBD approach.

Scope and contribution. We provide an effective combination of exact and metaheuristic methods for PDPT, hence consolidating a currently fragmented literature. Our LBBD is a more scalable exact method, as prior work [9, 24, 17] shows that synchronisation constraints, together with the large number of variables, severely limits the effectiveness of monolithic MILPs. We employ a strong relaxation serving as a master problem, which constructs parallel paths from the pickup to the delivery point of all requests. Most properties of the PDPT - the capacity constraints and precedences of pickup and delivery operations - are satisfied with significantly fewer variables, thus leaving vehicle assignment to routes plus vehicle synchronisation to an easy to solve subproblem. The use of additional valid inequalities and standard combinatorial (i.e., Benders') cuts improve the convergence to optimality.

Our LNS-based method emphasises simplicity, robustness, and minimal parameter tuning. Building on the LNS framework of [22], our key contributions include a parameter-free destroy operator for adaptive request removal, inspired by [10], a normalised insertion difficulty metric [10] that enhances the repair process without instance-specific tuning, and a combination of a randomised cheapest-insertion heuristic [6] with constant-time feasibility checks [18]. Together with a delayed acceptance mechanism [5] that improves solution quality, these yield an effective approach for PDPT, either to support LBBD or as standalone for large instances.

Given the scarcity of PDPT benchmarks, we adapt datasets from more restricted variants. Let us recall

that benchmarks from [17] have up to 30 requests, 3 transfer points, and 3 vehicles for loose time windows and only up to 5 requests for strict time windows. The most extensive dataset from the metaheuristics' community covers up to 100 requests [27], yet these instances are too specific for crowd-sourced drivers (e.g., they assume unlimited number of vehicles and neutralised capacity constraints).

To bridge the gap in benchmark availability, we develop a new instance generator inspired by [28]. Our generator introduces more robust scenarios by varying the tightness of time windows and incorporating transfer point construction. The generated instances present significant challenges for an exact method, whereas the LNS algorithm consistently produces high-quality solutions. To evaluate these solutions, we use the LBBB scheme as a lower bound provider, demonstrating that the LNS solutions accomplish an up to 40% optimality gap. The LBBB struggles to find feasible solutions when used alone (under a 1-hour time limit), but speeds up once starting from the LNS solution (which it often improves).

Outline. This paper describes a comprehensive toolbox of exact and metaheuristic methods for the PDPT, hence its remainder is organised as follows. Section 2 overviews our methodological framework and formally introduces PDPT along with our notation. Section 3 outlines the intuition of our LBBB and details the decomposition scheme. Section 4 elaborates on our refined LNS. The benchmark generator and experimental results are discussed in Section 5. Section 6 summarises our findings to motivate further work.

2 Preliminaries

2.1 Logic-Based Benders Decomposition

LBBB [14] is an extension of the classical Benders Decomposition [4]. Let $\mathcal{P}$ be a minimisation problem of two groups of variables x, y in domains D_x and D_y :

$$\mathcal{P} : \quad \min\{f(x) + g(y) : C_1(x), C_2(y), C_3(x, y), x \in D_x, y \in D_y\}.$$

Cost functions $f(x)$ and $g(y)$ are assumed to be non-negative. The problem is subject to constraints $C_1(x)$, $C_2(y)$ and $C_3(x, y)$. To partition $\mathcal{P}$, $\mathcal{M}$ is defined as

$$\mathcal{M} : \quad \min\{z : C_1(x), x \in D_x, z \geq f(x)\}.$$

As $f(x)$ and $g(y)$ are non-negative, z is a lower bound of the objective function of $\mathcal{P}$. Also all constraints of $\mathcal{M}$ are also imposed to $\mathcal{P}$. Therefore, $\mathcal{M}$ is a *relaxation* of $\mathcal{P}$ called the *master problem*.

Given the optimal solution $\bar{x}$ of $\mathcal{M}$ and the respective objective value $\bar{z}$, the following problem $\mathcal{S}$ computes a local optimum of $\mathcal{P}$:

$$\mathcal{S} : \quad \min\{f(\bar{x}) + g(y) : C_2(y), C_3(\bar{x}, y), y \in D_y\}.$$

We notice that $\mathcal{S}$ is a *subproblem* of $\mathcal{P}$ that finds the optimal solution in the restricted search space defined by $\bar{x}$. If $C_3(\bar{x}, y)$ are satisfied for the given solution $\bar{x}$, the optimal objective value of $\mathcal{S}$, denoted as $\bar{\zeta}$, is an upper bound of the optimal objective value of $\mathcal{P}$. Hence, $\frac{\bar{\zeta} - \bar{z}}{\bar{\zeta}}$ defines a valid optimality gap.

That is, LBBB employs an iterative exchange of knowledge between $\mathcal{M}$ and $\mathcal{S}$: $\mathcal{M}$ provides a lower bound of $\mathcal{P}$ and a partial solution $\bar{x}$, and, then, $\mathcal{S}$ returns an upper bound which is the local optimum of the restricted search space. A valid combinatorial cut ensures that the master problem avoids repeating the

same partial solution at any future iteration. Eventually, the lower bound and the best found upper bound converge to identical values, thus reaching the global optimal solution of $\mathcal{P}$.

If $\mathcal{S}$ can provide a feasible solution, then the combinatorial cut is a logic constraint of the following form, called *optimality cut*:

$$\text{if } x = \bar{x} \rightarrow z \geq \bar{\zeta}.$$

On the contrary, if $\mathcal{S}$ is infeasible for a partial solution $\bar{x}$, then a logic constraint called *feasibility cut* is generated:

$$x \neq \bar{x}.$$

Both types of logic constraints can be linearised trivially, if variables x are binary.

A hybridisation of Branch-and-Cut algorithms with LBBD, namely *Branch-and-Check* [30], has shown faster convergence, specially for partitions that involve small subproblems. In this case, the master problem is not solved optimally in an iterative manner; instead, the branching tree is explored once, interrupting the solution at each integer node to retrieve the local upper bound or prune the branch in case of infeasibility.

2.2 Large Neighbourhood Search

LNS [22] is an iterative local search method that explores a large neighborhood of a given solution in each iteration. This neighborhood is implicitly defined through two solution modification phases: first, the solution is partially destroyed by removing a subset of the requests and, then, the removed requests are reinserted to construct a new solution. The new solution is evaluated against the current one, and accepted based on a predefined criterion.

Hence, the main LNS components are the destroy operator defining how requests are removed from given solution, the repair operator determining how the removed requests are reinserted, and the acceptance criterion governing whether the repaired solution is accepted. A stopping criterion is also necessary as LNS cannot guarantee convergence to optimality. The design of these components influences the exploration of the solution space, impacting both the efficiency and effectiveness of the method.

Algorithm 1 The LNS metaheuristic

```

1: function LNS( $G$ )
2:    $s \leftarrow \text{INITIAL-SOLUTION}(G)$ 
3:    $s^* \leftarrow s$ 
4:   while stopping criterion not met do
5:      $s' \leftarrow \text{LOCAL-SEARCH}(s)$ 
6:     if  $\text{ACCEPT}(s')$  then
7:        $s \leftarrow s'$ 
8:     end if
9:     if  $\text{cost}(s') < \text{cost}(s^*)$  then
10:       $s^* \leftarrow s'$ 
11:    end if
12:  end while
13:  return  $s^*$ 
14: end function

```

Algorithm 1 presents the LNS workflow in an abstract manner. The method begins with an initial solution s for the given problem instance, which is also stored as the best-known solution s^* . The algorithm

then enters the local search process, which continues until a termination criterion is met. In each iteration, the neighborhood of the current solution s is explored by sequentially applying a pair of destroy and repair operators. The resulting solution s' replaces the current solution, if it meets the acceptance criterion. Additionally, if s' has a lower cost than s^* , it is stored as the best solution found so far.

2.3 Notation

The PDPT is defined on a set of requests $\mathcal{R}$ for pick-up and delivery, a set of locations $\mathcal{J}$ and a fleet of vehicles $\mathcal{K}$. The set of locations $\mathcal{J}$ comprises all depots $\mathcal{D}$, transfer points $\mathcal{T}$ and demand points $\mathcal{C}$ (both pick-up and delivery), the three subsets not being necessarily disjoint. Each request $r \in \mathcal{R}$ has a demand q_r (e.g., weight, volume) and must be picked up from a specific location $p_r \in \mathcal{C}$ and delivered to another location $d_r \in \mathcal{C}$. The requests are fulfilled by a fleet of homogeneous vehicles $\mathcal{K}$, each with capacity Q . Each vehicle $k \in \mathcal{K}$ begins its route at an origin depot $o_k \in \mathcal{D}$ and completes it at a destination depot $e_k \in \mathcal{D}$.

Table 1: Sets and parameters

Sets		
$\mathcal{R}$		Requests
$\mathcal{J}$		Locations
$\mathcal{K}$		Vehicles
$\mathcal{T}$		Transfer points; $\mathcal{T} \subset \mathcal{J}$
$\mathcal{D}$		Depots; $\mathcal{D} \subset \mathcal{J}$
$\mathcal{C}$		Demand points; $\mathcal{C} \subset \mathcal{J}$
Parameters		
$p_r \in \mathcal{J}$	$r \in \mathcal{R}$	Pickup point of request r
$d_r \in \mathcal{J}$	$r \in \mathcal{R}$	Delivery point of request r
q_r	$r \in \mathcal{R}$	Demand of request r
Q		Capacity of vehicles
$o_k \in \mathcal{J}$	$k \in \mathcal{K}$	Origin depot of vehicle k
$e_k \in \mathcal{J}$	$k \in \mathcal{K}$	Destination depot of vehicle k
l_j	$j \in \mathcal{J}$	Earliest time of visit at j
u_j	$j \in \mathcal{J}$	Latest time of visit at j
t_{ij}	$i, j \in \mathcal{J}$	Travel time from i to j
c_{ij}	$i, j \in \mathcal{J}$	Distance from i to j

Notably, when a specific request r is loaded onto a vehicle k , it can then be unloaded at a subsequent stop by the same or a different vehicle. In the former case, both p_r and d_r are included in the same route, ensuring that p_r precedes d_r . In the second case, vehicles may utilise designated a transfer point in $\mathcal{T}$, where requests can be offloaded and, subsequently, picked up by other vehicles for delivery or further transport.

Each location $j \in \mathcal{J}$ is associated with a specific time window $[l_j, u_j]$, which defines the permissible time interval during which the location can be visited for pickup or delivery. Each pair of locations (i, j) has a travel time t_{ij} and a distance c_{ij} . For simplicity we assume zero service times, i.e., pickups and deliveries are instantaneous. This is at no loss of generality, as service time depend on the location alone hence can be added to the travel time from any other location.

The objective here is to determine a set of vehicle routes minimising total distance travelled, that serve all requests while adhering to the specified time windows and vehicle capacity constraints. Each route must start at some vehicle's origin depot and end at its designated destination depot.

3 Benders decomposition

3.1 An improved exact approach

Let us recall from [9], for self-sufficiency of our presentation, the following five key properties of PDPT: (pdpt1) each route must begin at the vehicle’s origin depot and end at its destination depot without forming subtours, (pdpt2) each pickup and delivery point must be visited exactly once, (pdpt3) pickups must always precede their corresponding deliveries, (pdpt4) load transfers between vehicles must be synchronised (i.e., the vehicle unloading a request must arrive at the transfer location before the vehicle loading it), and (pdpt5) capacity constraints must be maintained throughout a vehicle’s route. As we move beyond [9] we must introduce property (pdpt6): all visits must comply with predefined time windows. We refer to these properties in the sequel to explain the correctness of our mathematical modelling.

PDPT has been formulated as a MILP in [24] and [17]. A key limitation lies in the MILP’s reliance on four-indexed binary variables y_{rijk} , which are set to 1 if request r is loaded onto vehicle k while traveling from location i to j . Given that the total number of locations $\mathcal{J}$ includes pickup and delivery points for each request, origin and destination depots for each vehicle, and all transfer points ($2 \cdot |\mathcal{R}| + 2 \cdot |\mathcal{K}| + |\mathcal{T}|$ locations), the number of such variables explodes even for moderate instances.

Partitioning the MILP can facilitate the computation of feasible solutions. The work of [9] already considers a relaxation of the holistic model by omitting complicating constraints, and a feasibility subproblem to validate if the obtained partial solution can lead to a feasible routing schedule. Combinatorial cuts [7] are added to the relaxation, if a feasible solution is not obtained. This decomposition has not shown increased scalability, but accelerates the optimal solution of small-scaled instances (up to 6 requests).

We therefore introduce a novel partitioning strategy, leading to a stronger relaxation of the original problem as the master problem. Specifically, we exclude from the master problem the set of vehicles $\mathcal{K}$ and the synchronisation of transfers (pdpt4). The intuition behind this choice is to enforce properties (pdpt2), (pdpt3), and (pdpt5) while relaxing (pdpt1) and (pdpt6) in a way that increases the likelihood of generating a feasible routing plan. The solution to the master problem consists of a set of parallel paths, each corresponding to a request $r \in \mathcal{R}$. These paths inherently satisfy (pdpt2), (pdpt3), and (pdpt5), thus allowing a simpler subproblem to focus on assigning selected path edges to valid routes, while addressing the remaining properties.

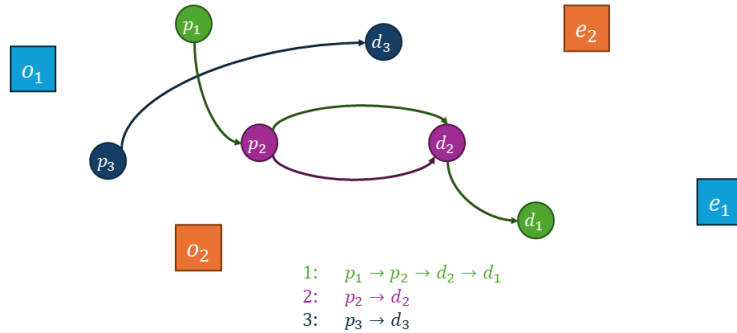


Figure 1: Example of path construction for three requests and two vehicles

Figure 1 illustrates an example involving three requests of size 1 and two vehicles with a capacity of

2. Each request r has each own path, beginning from p_r and ending at d_r . The paths of requests 1 and 2 overlap, as they share trip $p_2 \rightarrow d_2$ (hence the two parallel edges), whereas the path for request 3 is disjoint. Importantly, incorporating this partial solution into the original problem, while preserving the path edges, significantly simplifies the optimisation problem. Additionally, several constraints of the original problem are inherently satisfied by this partial solution: vehicle capacities are not exceeded for any path, and the precedence of pickup and delivery operations is maintained. Such a construction of paths is clearly a relaxation of the original problem, as it omits several constraints, most notably the involvement of vehicles.

3.2 The master problem

The master MILP. The formulation relies on two sets of binary variables. Variables y_{rij} are set to 1 if request $r \in \mathcal{R}$ is loaded (on some vehicle) during trip (i.e., edge) (i, j) , where $i, j \in \mathcal{J}$, and 0 otherwise. Variables x_{ij} indicate whether trip (i, j) is selected or not. Let us now explain why the above formulation is correct by associating constraints with the properties (pdpt1)-(pdpt6) listed at the end of Section 2.1.

min z

$$z \geq \sum_{i \in \mathcal{J}} \sum_{j \in \mathcal{J}} c_{ij} \cdot x_{ij} \quad (1)$$

$$\sum_{j \in \mathcal{J}} y_{rp_r j} = 1 \quad r \in \mathcal{R} \quad (2)$$

$$\sum_{j \in \mathcal{J}} y_{r j d_r} = 1 \quad r \in \mathcal{R} \quad (3)$$

$$\sum_{i \in \mathcal{J}} y_{rij} \leq 1 \quad r \in \mathcal{R}, j \in \mathcal{J} \notin \{p_r, d_r\} \quad (4)$$

$$\sum_{i \in \mathcal{J}} y_{rij} = \sum_{i \in \mathcal{J}} y_{rji} \quad r \in \mathcal{R}, j \in \mathcal{J} \notin \{p_r, d_r\} \quad (5)$$

$$\sum_{i \in \mathcal{J}} x_{ij} = 1 \quad j \in \mathcal{C} \quad (6)$$

$$\sum_{i \in \mathcal{J}} x_{ij} = \sum_{i \in \mathcal{J}} x_{ji} \quad j \in \mathcal{J} \setminus \mathcal{D} \quad (7)$$

$$\sum_{r \in \mathcal{R}} q_r \cdot y_{rij} \leq Q \cdot x_{ij} \quad i \in \mathcal{J}, j \in \mathcal{J} \quad (8)$$

$$[\text{Valid inequalities}] \quad (9)$$

$$[\text{Time windows constraints}] \quad (10)$$

$$[\text{Benders cuts}] \quad (11)$$

$$x_{ij} \in \{0, 1\} \quad i \in \mathcal{J}, j \in \mathcal{J}$$

$$y_{rij} \in \{0, 1\} \quad r \in \mathcal{R}, i \in \mathcal{J}, j \in \mathcal{J}, i \neq j$$

The objective function is the total distance travelled (1). Constraints (2) and (3) ensure that each request is loaded on some edge departing from its pickup point and on some edge arriving at its delivery point; notice that this also ensures that property (pdpt3) is eventually satisfied. Constraints (4) guarantee that each request visits any location at most once; if a transfer point is eligible for multiple visits, the set $\mathcal{T}$

should include duplicate copies of that location, as also noted in [9]. Constraints (5) preserve the flow, i.e., they ensure that each request departs from any visited location, except for its pickup and delivery points.

Regarding now variables x_{ij} , Constraints (6) enforce that each demand point is visited once, while Constraints (7) ensure flow conservation at non-depot locations: these constraints satisfy property (pdpt2), while also enforcing (pdpt1). Constraints (8) serve a dual purpose: they enforce that each trip carries a request volume within the vehicle capacity and establish the connection between variables y_{rij} and x_{ij} . Since the vehicles have identical capacity, Constraints (8) satisfy property (pdpt5).

Valid inequalities. Constraints (2)–(8) capture most of the PDP intricacies. Notice, however, that Constraints (6) do not involve transfer points and allow multiple incoming and outgoing flows through locations in $\mathcal{T}$. To ensure the validity of paths from p_r to d_r for all requests $r \in \mathcal{R}$, eliminate invalid trips, and strengthen the MILP lower bound, we also incorporate the following set of valid inequalities.

$$x_{jj} = 0 \quad j \in \mathcal{J} \quad (12)$$

$$x_{d_r p_r} = 0 \quad r \in \mathcal{R} \quad (13)$$

$$x_{o_k d_r} = 0 \quad k \in \mathcal{K}, r \in \mathcal{R} \quad (14)$$

$$x_{p_r e_k} = 0 \quad k \in \mathcal{K}, r \in \mathcal{R} \quad (15)$$

$$x_{o_k e_{k'}} = 0 \quad k \in \mathcal{K}, k' \in \mathcal{K}, k \neq k' \quad (16)$$

$$x_{j o_k} = 0 \quad k \in \mathcal{K}, j \in \mathcal{J} \quad (17)$$

$$x_{e_k j} = 0 \quad k \in \mathcal{K}, j \in \mathcal{J} \quad (18)$$

$$\sum_{i \in \mathcal{J}} y_{rij} = 0 \quad r \in \mathcal{R}, j \in \mathcal{D} \quad (19)$$

$$\sum_{j \in \mathcal{J}} y_{rd_r j} = 0 \quad r \in \mathcal{R} \quad (20)$$

$$\sum_{j \in \mathcal{J}} y_{rj p_r} = 0 \quad r \in \mathcal{R} \quad (21)$$

$$\sum_{j \in \mathcal{J}} x_{o_k j} = 1 \quad k \in \mathcal{K} \quad (22)$$

$$\sum_{j \in \mathcal{J}} x_{j e_k} = 1 \quad k \in \mathcal{K} \quad (23)$$

Constraints (12)–(21) eliminate a significant number of variables, thereby reducing the search space. Specifically, Constraints (12) prohibit loops, while (13) prevent trips from a delivery point back to the corresponding pickup point. Constraints (14) and (15) ensure that no delivery point is the first stop, and no pickup point is the last stop of any vehicle.

Although direct connections between an origin and a destination depot are not explicitly forbidden - since such trips indicate that a vehicle remains unused - the connection of an origin depot with the destination depot of a different vehicle is invalid, hence imposing (16). Origin depots cannot serve as destinations, as imposed by (17), and destination depots cannot serve as origins, as dictated by (18). Depot locations must not be included in the path of any request, as stated by (19).

Constraints (20) and (21) ensure, respectively that requests cannot be loaded before reaching their pickup point and they remain loaded until reaching their delivery point. Furthermore, (22) and (23) guarantee that

depots are incorporated into the routing plan.

The following inequalities reduce the likelihood of partial solutions that lead to infeasibility:

$$\sum_{i \in \mathcal{J}} x_{ij} \leq |\mathcal{K}| \quad j \in \mathcal{T}. \quad (24)$$

Let us explain this. As (6) do not apply to $\mathcal{T}$, there are no restrictions on assigning transfer points to trips. However, each vehicle can visit a transfer point at most once, thus we should prevent an overuse of edges accessing the same transfer point. Constraints (24) limit the number of visits per transfer point to $|\mathcal{K}|$, hence ensuring that the selected edges can be validly assigned to the vehicles in the subproblem presented next.

We note that limiting the number of visits to one per (vehicle, transfer point) pair is a modeling decision rather than a strict constraint, as transfer points might be used without such a restriction. To accommodate multiple visits, the set $\mathcal{T}$ should include duplicate copies of transfer points, allowing them to be visited more often (a similar approach is suggested by [9]).

Time windows constraints. Time windows impose additional constraints that account for travel times along selected trips. Therefore, we introduce two sets of continuous variables. The variables a_j represent the arrival times at all locations except transfer points. Since transfer points can be visited by multiple vehicles, their arrival times cannot be uniquely defined. Therefore, we introduce variables b_{rj} , which indicate the arrival time of request r at transfer point j . All arrival time variables are constrained within the time window limits l_j and u_j . The constraints are as follows.

$$a_i + t_{ij} - a_j \leq (t_{ij} + u_i) \cdot (1 - x_{ij}) \quad \forall i \in \mathcal{J} \setminus \mathcal{T}, j \in \mathcal{J} \setminus \mathcal{T} \quad (25)$$

$$a_i + t_{ij} - b_{rj} \leq (t_{ij} + u_i) \cdot (1 - y_{rij}) \quad \forall r \in \mathcal{R}, i \in \mathcal{J} \setminus \mathcal{T}, j \in \mathcal{T} \quad (26)$$

$$b_{ri} + t_{ij} - a_j \leq (t_{ij} + u_i) \cdot (1 - y_{rij}) \quad \forall r \in \mathcal{R}, i \in \mathcal{T}, j \in \mathcal{J} \setminus \mathcal{T} \quad (27)$$

$$b_{ri} + t_{ij} - b_{rj} \leq (t_{ij} + u_i) \cdot (1 - y_{rij}) \quad \forall r \in \mathcal{R}, i \in \mathcal{T}, j \in \mathcal{T} \quad (28)$$

$$l_j \leq a_j \leq u_j \quad j \in \mathcal{J}$$

$$l_j \leq b_{rj} \leq u_j \quad r \in \mathcal{R}, j \in \mathcal{T}$$

Constraints (25) determine the arrival times of non-transfer points (interestingly, similar constraints are commonly used for computing completion times of operations which are determined by sequence-dependent times). Constraints (26)-(28) define the arrival times of trips which involve transfer points: since each transfer point can be visited for more than once, a single-index variable a_t for each $t \in \mathcal{T}$ would be invalid.

3.3 The subproblem

Preprocessing. For any solution of the master problem, let $\bar{y}_{rij}$ be the values of variables y_{rij} . The following simple procedure constructs the path of each request $r \in \mathcal{R}$:

Algorithm 2 Construction of path for request r

```
1: Set  $\text{path}_r = \{p_r\}$  be the path of  $r$  and  $\lambda = p_r$  be the last visited location of the path;  
2: while  $\lambda \neq d_r$  do  
3:   for  $j \in \mathcal{J}$  do  
4:     if  $\bar{y}_{r\lambda j} = 1$  then  
5:       Append  $j$  to  $\text{path}_r$ ;  
6:        $\lambda \leftarrow j$ ;  
7:       break  
8:     end if  
9:   end for  
10: end while  
11: Return  $\text{path}_r$ ;
```

Fixing the selected trips in paths leads to a subproblem, in which a valid vehicle routing plan must be generated, while adhering to the paths determined by the master problem. Since capacity constraints and the precedence of pickups and deliveries are already ensured, the focus shifts to synchronising operations at transfer points while respecting time windows for all locations.

To achieve this synchronisation, we construct vectors τ_r for each request r , containing triples of consecutive locations in path_r , where the intermediate location is a transfer point. For example, if trips (i, t) and (t, j) are included in path_r and t is a transfer point, then the triple (i, t, j) is appended to τ_r . We also define set $\mathcal{E}$, containing all trips which are included in any path: $\mathcal{E} = \{(i, j) : \sum_{r \in \mathcal{R}} \bar{y}_{rij} \geq 1\}$.

MILP formulation. The subproblem is formulated as an MILP and aims to assemble the paths from the master problem into complete routes for the available vehicles $\mathcal{K}$. This is done while respecting the time windows of all requests and ensuring synchronisation between drop-offs and pickups at transfer points. The subproblem is required to retain all trips (i, j) included in the paths of requests $\mathcal{R}$ and is permitted to

generate additional trips that connect disjoint paths into a single route.

$$\begin{aligned}
\min \quad & \sum_{i \in \mathcal{J}} \sum_{j \in \mathcal{J}} \sum_{k \in \mathcal{K}} c_{ij} \cdot x_{ijk} \\
& \sum_{j \in \mathcal{J}} x_{o_k j k} = 1 & k \in \mathcal{K} & \quad (29) \\
& \sum_{j \in \mathcal{J}} x_{j e_k k} = 1 & k \in \mathcal{K} & \quad (30) \\
& \sum_{j \in \mathcal{J}} x_{j o_k k} = 0 & k \in \mathcal{K} & \quad (31) \\
& \sum_{j \in \mathcal{J}} x_{e_k j k} = 0 & k \in \mathcal{K} & \quad (32) \\
& \sum_{i \in \mathcal{J}} x_{ijk} \leq 1 & j \in \mathcal{J}, k \in \mathcal{K} & \quad (33) \\
& \sum_{i \in \mathcal{J}} x_{ijk} = \sum_{i \in \mathcal{J}} x_{jik} & j \in \mathcal{J} \setminus \mathcal{D}, k \in \mathcal{K} & \quad (34) \\
& \sum_{j \in \mathcal{J}} x_{p_r j k} = 1 & r \in \mathcal{R}, k \in \mathcal{K} & \quad (35) \\
& \sum_{j \in \mathcal{J}} x_{j d_r k} = 1 & r \in \mathcal{R}, k \in \mathcal{K} & \quad (36) \\
& \sum_{k \in \mathcal{K}} x_{ijk} = 1 & (i, j) \in \mathcal{E} & \quad (37) \\
& a_{ik} + t_{ij} - a_{jk} \leq (t_{ij} + u_i) \cdot (1 - x_{ijk}) & i \in \mathcal{J}, j \in \mathcal{J}, k \in \mathcal{K} & \quad (38) \\
& a_{p_r k} \leq a_{d_r k} & r \in \mathcal{R}, k \in \mathcal{K} & \quad (39) \\
& a_{tk} - a_{tl} \leq u_t \cdot (2 - x_{itk} - x_{tjl}) & r \in \mathcal{R}, (i, t, j) \in \tau_r, k \in \mathcal{K}, l \in \mathcal{K} & \quad (40) \\
& x_{ijk} \in \{0, 1\} & i \in \mathcal{J}, j \in \mathcal{J}, i \neq j, k \in \mathcal{K} & \\
& l_j \leq a_{jk} \leq u_J & j \in \mathcal{J}, k \in \mathcal{K} &
\end{aligned}$$

By means of an example, if the solution to the master problem were as illustrated in Figure 1, the subproblem would be compelled to generate a single route for the paths of Requests 1 and 2, as they share the trip (p_2, d_2) . Additionally, it would determine whether the path for Request 3 should be incorporated into the same route or assigned to a different vehicle.

Variables x_{ijk} extend the functionality of the relaxed variables x_{ij} from the master problem by incorporating the set of vehicles. If vehicle k includes trip (i, j) in its route, then x_{ijk} is set to 1; otherwise, it remains 0. Additionally, continuous variables a_{jk} represent the arrival times of vehicle k at location j .

Each vehicle starts at its origin depot and returns to its destination depot, as enforced by Constraints (29)–(32). Each location can be visited at most once, hence imposing (33), while flow conservation is maintained at all non-depot locations as designated by (34). Constraints (35) and (36) ensure that all pickup and delivery points are visited. Moreover, Constraints (37) guarantee that all trips selected in the master problem are included in the solution.

Arrival times at visited locations are defined by Constraints (38), while (39) enforce the correct order of pickup and delivery for each request. Finally, (40) ensure vehicle synchronisation at transfer points: a

vehicle picking up a request must arrive at the transfer point after the vehicle dropping it off.

3.4 The Branch-and-Check method

Recall that $\mathcal{P}$ denotes the original PDPT and $\mathcal{M}$ the master problem in the decomposition. Solving $\mathcal{M}$ generates solutions for iterations $n = 1, 2, \dots$, each associated with a set of selected edges $\mathcal{E}^n$ and a vector $\tau^n = \{\tau_r^n : r \in \mathcal{R}\}$. The corresponding subproblem $\mathcal{S}(\mathcal{E}^n, \tau^n)$ then constructs a complete routing plan, yielding a feasible solution to $\mathcal{P}$ as long as the fixed edges permit it. The objective value of the optimal solution of $\mathcal{M}$ is a lower bound of the objective value of the optimal solution of $\mathcal{P}$, while the objective value of the subproblem $\bar{\zeta}^n$ at any iteration n is an upper bound. We solve the master problem $\mathcal{M}$ through Branch-and-Check, i.e., whenever an MILP solver finds an integer solution to $\mathcal{M}$, the solution process is interrupted, and the corresponding subproblem evaluates feasibility.

If the subproblem is feasible, let $\bar{\zeta}^n$ denote its objective value: the following optimality cut ensures that if the master problem attempts to reproduce the same solution, its objective value is constrained to the actual upper bound $\bar{\zeta}^n$:

$$z \geq \bar{\zeta}^n - \bar{\zeta}^n \cdot (|\mathcal{E}^n| - \sum_{(i,j) \in \mathcal{E}^n} x_{ij}). \quad (41)$$

Otherwise, i.e., if the selected edges $\mathcal{E}^n$ fail to produce a feasible routing plan, we add to $\mathcal{M}$ the standard feasibility cut

$$\sum_{(i,j) \in \mathcal{E}^n} x_{ij} \leq |\mathcal{E}^n| - 1. \quad (42)$$

After exploring the solution at iteration n , the solution process for $\mathcal{M}$ continues. If the latest upper bound improves upon all previously found bounds, the best lower and upper bounds are combined to update the optimality gap. Once the gap reaches zero, the routing plan corresponding to the best upper bound is retrieved, representing the global optimal solution of $\mathcal{P}$.

The validity of (41) and (42) is straightforward. Regarding (41), a partial solution that replicates the input to the subproblem from iteration n (i.e., maintaining the same paths for all requests) results in an objective value z refined to the corresponding upper bound $\bar{\zeta}^n$. Any deviation in the paths neutralises the effect of cut (41), thus ensuring its validity. Similarly, the feasibility cut (42) guarantees that at least one edge from any previously retrieved set of paths is eliminated. We add cuts at any node in which an integer solution is found and maintain cuts in all subsequent nodes.

4 Large Neighbourhood Search

We propose a metaheuristic methodology that builds upon and enhances existing techniques. Our aim is to develop a simple yet effective method, ensuring that solution quality is not heavily dependent on the specific characteristics of each problem instance. Following this aim, we propose a method based on LNS that employs a single combination of destroy and repair operators. Unlike existing ALNS techniques, our approach eliminates the need for the parameter tuning associated with the adaptive selection of destroy and repair mechanisms. Furthermore, we introduce targeted modifications to widely-used destroy and repair

operators to enhance the robustness of our method. Our computational experiences appears to support our adaptation (see Section 5).

4.1 Destroy operator

Degree of destruction. The degree of destruction applied to the solution is a crucial LNS parameter [22]. If it is too small, the neighborhood defined by the destroy operator becomes narrow, restricting the exploration of the search space. Conversely, if it is too large, the local search process may resemble a re-optimisation procedure rather than an effective exploration strategy. To determine this, we adopt the approach of [25], where the number of removed requests is randomly selected within a predefined range.

Request selection criterion. For the selection criterion of the requests to be removed, we utilise a modified version of the related removal operator proposed in [10]. The underlying principle of this operator is to remove requests that exhibit low dissimilarity, as this increases the likelihood of finding a beneficial reinsertion that improves the overall solution cost [26].

Let q_r be the demand associated with request r . Let p_r and d_r be the pickup and delivery node associated with r , respectively. Let l_i be the earliest possible service time for a node i . Let t_{ij} be the travel time associated with traveling from i to j . The dissimilarity between requests r and r' in the related removal operator is defined as:

$$D(r, r') = \theta_1 \cdot |q_r - q_{r'}| + \theta_2 \cdot (t_{p_r p_{r'}} + t_{d_r d_{r'}}) + \theta_3 \cdot (|l_{p_r} - l_{p_{r'}}| + |l_{d_r} - l_{d_{r'}}|),$$

where θ_1 , θ_2 and θ_3 are parameters to be tuned, primarily used to handle the potentially different scales of demands, travel times and time windows.

Our proposed modification refines the computation of request dissimilarity, eliminating the need for parameter tuning and thereby simplifying the algorithm. Specifically, each request r is represented as a feature vector

$$vec(r) = \begin{bmatrix} q_r & x(p_r) & y(p_r) & x(d_r) & y(d_r) & l_{p_r} & l_{d_r} & st(p_r) & st(d_r) \end{bmatrix}^T,$$

where $x(i)$ and $y(i)$ are the coordinates of node i respectively, and $st(i)$ is its service time.

Dissimilarity between requests is measured using the Mahalanobis distance between their vector representations. Mahalanobis distance eliminates the influence of scale, while also accounting for correlations between features, preventing the over-weighting of redundant information.

4.2 Repair operator

Insertion order. To determine the order in which the non-serviced requests are to be considered, we use a modified version of the ‘Insertion Ease’ metric proposed by [10]. The key idea is that requests that are hard with respect to the problem’s constraints, should be inserted in the partial solution first. The Insertion Ease of a request r is defined as follows:

$$IE(r) = \theta_1 \cdot q_r + \theta_2 \cdot t_{p_r d_r} - \theta_3 \cdot (u_{p_r} - l_{p_r} + u_{d_r} - l_{d_r}),$$

where u_i is the latest possible time to begin service at node i , and θ_1 , θ_2 and θ_3 are parameters to be tuned, primarily used to handle the potentially different scales of demands, travel times and time windows.

To enhance the robustness of this approach and avoid problem-specific parameter tuning, we propose a modified formula to compute the insertion difficulty of each request. Specifically, we first compute the minimum and maximum of the following quantities among the instance’s requests: demand q_r , pickup time window width $w(p_r)$, delivery time window width $w(d_r)$, pickup service time $st(p_r)$, delivery service time $st(d_r)$, and travel time between the pickup and delivery node $t_{p_r d_r}$. We define the Insertion Difficulty of request r as follows:

$$ID(r) = \bar{q}_r + \bar{t}_{p_r d_r} + \bar{st}(p_r) + \bar{st}(d_r) - \bar{w}(p_r) - \bar{w}(d_r),$$

where $\bar{x}$ denotes that min-max scaling was applied to the quantity x . We sort the unserved requests in a decreasing Insertion Difficulty order.

Request insertion. For the insertion of each request r in the partial solution s^- , we use a randomized cheapest-insertion heuristic. That is, every possible insertion of r is evaluated, with or without the use of a transfer point t , leading to a complexity of $O(|\mathcal{T}| \cdot |\mathcal{K}|^2 \cdot |\mathcal{R}|^4)$, which is far from negligible. Checking the feasibility of each candidate insertion is done through the method proposed by [18], i.e., using constant-time feasibility testing. Now, to introduce randomness to the insertion procedure, which has proven to be advantageous for the optimisation process [22], we employ the technique proposed by [6]. Specifically, each feasible candidate insertion is omitted with a probability β . Thus, the insertion heuristic returns the cheapest feasible insertion not-skipped by the randomisation process.

4.3 Acceptance criterion

Allowing the consideration of non-improving solutions has been widely recognized for its advantages and has become an essential component of many local search techniques. Although Simulated Annealing [15] and its variants are relevant acceptance mechanisms, they require extensive parameter tuning that might become highly problem-specific and hence significantly impact performance. We therefore adopt an alternative acceptance criterion of [5] to minimise tuning and, hopefully, remain robust across various instances.

Specifically, we accept a new solution if it is superior to the current one or, if worse, provided that it is better than the solution from L iterations prior. By incorporating this delayed acceptance mechanism, we allow temporary degradations in solution quality to escape local optima. The algorithm maintains a historical record of past solutions, iteratively updating it to guide the acceptance of new candidates, thereby improving exploration while requiring minimal parameterisation.

5 Experiments

All experiments run on a server equipped with 32 AMD Ryzen Threadripper PRO 5955WX 16-core processors and 32 GB of RAM, running Ubuntu 22.04.5 LTS. To solve the Branch-and-Check algorithm, we employ the *Gurobi 11.0.3* optimiser via the open-source optimisation library *Pyomo 6.8.0* in *Python 3.10.12*. The Large Neighborhood Search algorithm was implemented in C# using the .NET 8.0 SDK and runtime.

5.1 Literature instances

Loose Time Windows. The first group of instances assumes a universal time window of $[0, 999]$ for all requests, where the upper bound imposes the use of multiple vehicles for instances exceeding 20 requests. Instances with fewer than 10 requests are not discussed, as both our LBBB and the MILP [17] find the optimal solution in approximately one second. The tested instances include $|\mathcal{R}| = 10, 12, 15, 20, 25, 30$ requests, with $|\mathcal{K}| = 2, 3$ vehicles and $|\mathcal{T}| = 1, 2, 3$ transfer points. For cases where $|\mathcal{K}| = 3$, the number of transfer points is fixed at $|\mathcal{T}| = 3$. Per each combination, we report averages over 10 randomly generated datasets.

Table 2 presents a comparative summary of the average performance of our LBBB method against the MILP [17] within the time limit of one hour. The ‘MILP [17]’ set of columns reports the average values of the Lower Bound (LB), Upper Bound (UB), optimality Gap (%), and computational time (Time), but only for instances where a feasible solution was found; the number of such instances is shown in the ‘Feasible’ column. As LBBB finds solutions in more instances than the MILP, ‘LBBB - comparison’ provides the corresponding results for the LBBB method but only for the instances where the MILP found a feasible solution. Last, ‘LBBB - for all instances’ presents the results obtained for LBBB over all instances for which it found a feasible solution.

Table 2: Comparison between the MILP [17] and the LBBB - loose time windows

$ \mathcal{R} $	$ \mathcal{K} $	$ \mathcal{T} $	MILP [17]					LBBB - comparison				LBBB - for all instances				
			LB	UB	Gap (%)	Time (s)	Feasible	LB	UB	Gap (%)	Time (s)	LB	UB	Gap (%)	Time (s)	Feasible
10	2	1	717.27	717.27	0.00	11	10	717.27	717.27	0.00	3	717.27	717.27	0.00	3	10
10	2	2	703.04	703.04	0.00	21	10	703.04	703.04	0.00	10	703.04	703.04	0.00	9	10
10	3	3	686.75	686.75	0.00	8	10	686.75	686.75	0.00	26	686.75	686.75	0.00	25	10
12	2	1	729.06	729.06	0.00	43	10	729.06	729.06	0.00	3	729.06	729.06	0.00	3	10
12	2	2	751.60	751.60	0.00	72	10	751.60	751.60	0.00	11	751.60	751.60	0.00	11	10
12	3	3	812.36	812.36	0.00	218	10	812.36	812.36	0.00	154	812.36	812.36	0.00	154	10
15	2	1	933.01	933.01	0.00	362	10	933.01	933.01	0.00	13	933.01	933.01	0.00	13	10
15	2	2	923.66	923.66	0.00	108	10	923.66	923.66	0.00	24	923.66	923.66	0.00	23	10
15	3	3	946.38	948.08	0.19	589	10	948.08	948.08	0.00	442	948.08	948.08	0.00	442	10
20	2	1	1135.55	1163.69	2.81	1985	9	1154.80	1154.80	0.00	143	1157.03	1157.03	0.00	178	10
20	2	2	1144.24	1155.54	0.99	2292	6	1154.56	1154.56	0.00	271	1116.39	1128.70	1.10	969	10
20	3	3	1083.58	1186.04	8.14	2616	9	1113.37	1114.55	0.12	938	1103.19	1105.76	0.25	1204	10
25	2	1	1381.81	1439.50	4.26	2393	2	1416.85	1416.85	0.00	40	1291.99	1329.76	3.29	1852	10
25	2	2	1279.85	1395.43	8.49	3422	2	1308.23	1329.79	1.75	1940	1289.43	1312.19	1.72	2572	10
25	3	3	1337.46	1506.54	11.23	3600	3	1376.36	1376.36	0.00	777	1311.04	1326.76	1.26	1470	10
30	2	1	1412.27	1574.83	10.32	3600	1	1454.08	1463.82	0.67	3600	1534.16	1596.54	3.99	2985	10
30	2	2	-	-	-	-	0	-	-	-	-	1425.03	1508.49	5.71	3376	7
30	3	3	-	-	-	-	0	-	-	-	-	1479.26	1560.68	5.53	2943	9

For instances with 10 and 12 requests, both methods solve all datasets to optimality within seconds, making the comparison inconclusive. Similarly, for 29 out of 30 instances with 15 requests, both approaches yield optimal solutions. The only exception is a dataset with 3 vehicles and 3 transfer points, which the MILP fails to solve optimally - hence, the average optimality gap for the MILP is slightly above zero.

Nevertheless, for larger instances, the LBBB consistently outperforms the MILP in both lower and upper bounds. As a result, the LBBB maintains a notably smaller average optimality gap, while the MILP’s performance deteriorates. Additionally, the LBBB produces feasible solutions in 16 out of 18 instance groups at an average gap of at most 4%. In the remaining two groups, a few instances remain unsolved for LBBB within one hour, whereas MILP solves none. Detailed results are displayed in Table 9 of the Appendix.

Next, for the larger datasets of [17] that include 20-30 requests, the LNS algorithm is applied first to generate a feasible solution. This solution is then used as a warm-start for the master problem in the Branch-and-Check, allowing LBBB to explore a branching tree that originates from a feasible node. Table 3 presents relevant results for three approaches: the Branch-and-Check algorithm (‘LBBB’), the LNS algorithm (‘LNS’),

and the Branch-and-Check algorithm initialised with a warm-start solution from LNS ('LBBD⁺'). Each row reports average values across 10 instances of the same number of requests, vehicles, and transfer points. It is worth noting that for four specific instances - three with 30 requests, 2 vehicles, and 2 transfer points, and one with 30 requests, 3 vehicles, and 3 transfer points - the standalone LBBD algorithm failed to find a feasible solution within the time limit. Consequently, these instances are excluded from the average values in the 'LNS' and 'LBBD⁺' columns to enable comparison, despite both methods successfully identifying feasible solutions. Table 10 of the Appendix presents the results in detail.

For the LNS results, the column 'Err' refers to the average deviation between the upper bounds of LNS and LBBD, computed as $\text{Err} = \frac{\text{UB}(\text{LNS}) - \text{UB}(\text{LBBD})}{\text{UB}(\text{LNS})} \%$. Column 'Gap' reports the optimality gap, defined as the relative difference between the upper bound of LNS and the best lower bound, as provided consistently by 'LBBD⁺'. These metrics demonstrate that LNS produces near-optimal solutions in negligible time, thus being an effective warm-start approach. The benefit of warm-starting is further highlighted in columns 'Avg Imp' and 'Avg Acc'. 'Avg Imp' reflects the average improvement in optimality gap for instances that could not be solved to optimality by 'LBBD': $\text{Avg Imp} = \text{Gap}(\text{LBBD}) - \text{Gap}(\text{LBBD}^+) \%$. Meanwhile, 'Avg Acc' captures the average time savings for instances successfully solved to optimality by both methods: $\text{Avg Acc} = \text{Time}(\text{LBBD}) - \text{Time}(\text{LBBD}^+)$. Note that 'Avg Imp' is not reported for instance groups where all 10 cases are solved to optimality (i.e., for 20 requests, 2 vehicles and 1 transfer point). Overall, the results underscore the value of integrating LNS as a warm-start mechanism within the LBBD framework.

Table 3: Average results of LNS and LBBD with or without a warm-start solution

$ \mathcal{R} $	$ \mathcal{K} $	$ \mathcal{T} $	LBBD				LNS				LBBD ⁺				Avg Imp	Avg Acc
			LB	UB	Gap	Time	UB	Err	Gap	Time	LB	UB	Gap	Time		
20	2	1	1157.03	1157.03	0.00	179	1199.12	3.47	3.47	1	1157.03	1157.03	0.00	129	-	50
20	2	2	1116.39	1128.70	1.10	969	1173.72	3.81	4.83	1	1117.05	1128.26	1.00	907	0.49	77
20	3	3	1103.19	1105.76	0.25	1204	1156.84	4.40	4.63	1	1103.37	1105.76	0.24	1144	0.09	76
25	2	1	1291.99	1329.76	3.29	1852	1387.14	3.92	6.61	1	1297.97	1323.58	2.24	1838	2.09	28
25	2	2	1289.43	1312.19	1.72	2573	1367.67	4.02	5.39	1	1293.43	1312.19	1.42	2429	0.51	361
25	3	3	1311.04	1326.76	1.26	1470	1391.22	4.65	5.73	1	1312.66	1326.05	1.08	1401	0.60	98
30	2	1	1534.16	1596.54	3.99	2985	1692.71	5.65	9.15	1	1539.56	1592.38	3.40	2888	0.85	323
30	2	2	1425.03	1508.49	5.71	3376	1602.89	5.96	10.89	1	1431.97	1493.79	4.32	3316	1.63	422
30	3	3	1479.26	1560.68	5.53	2944	1657.13	5.55	10.62	1	1484.55	1551.97	4.64	2925	1.15	197

Strict Time Windows. We consider three different time window spans of 180, 240, and 300 minutes. Within each span, three request generation scenarios are examined: 'L' (long distances between pickup and delivery points), 'S' (short distances), and 'M' (a mix of both). Instance generation follows the methodology of [27], with a small number of requests (3, 4, and 5 requests) and a fleet of 4 vehicles. Additionally, all datasets include 4 transfer points, while instances with 3 requests are also tested with 5 transfer points.

The results appear in Table 4. Column $|\mathcal{R}|$ denotes the number of requests, while column $|\mathcal{T}|$ indicates the number of transfer points. The time window span (180, 240, or 300 minutes) and distance type (L, M, or S) are indicated under 'TW'. The number of vehicles is omitted, as all datasets assume a fleet of 4 vehicles. Each row reports the average results for all datasets matching the specified attributes where both methods found feasible solutions.

In 34 out of 36 instance groups, both methods successfully find feasible solutions for all 10 instances. However, for the groups with 5 requests and the 180L and 180M configurations, neither method could solve all instances within one hour. Specifically, the reported averages for these groups include only 5-7 instances, as shown in columns 'Feasible' that indicate the number of instances for which a feasible solution is found.

Both methods achieve optimality for the vast majority of instances. For datasets with 3 and 4 requests,

Table 4: Comparison between the MILP [17] and the LBB - tight time windows

$ \mathcal{R} $	TW	$ \mathcal{T} $	MILP of [17]					LBB				
			LB	UB	Gap (%)	Time (s)	Feasible	LB	UB	Gap (%)	Time (s)	Feasible
3	180L	4	411.75	411.75	0.00	7	10	411.75	411.75	0.00	23	10
3	180M	4	330.92	330.92	0.00	8	10	330.92	330.92	0.00	18	10
3	180S	4	261.33	261.33	0.00	3	10	261.33	261.33	0.00	7	10
3	240L	4	376.67	376.67	0.00	4	10	376.67	376.67	0.00	10	10
3	240M	4	299.72	299.72	0.00	4	10	299.72	299.72	0.00	7	10
3	240S	4	251.64	251.64	0.00	2	10	251.64	251.64	0.00	2	10
3	300L	4	376.45	376.45	0.00	6	10	376.45	376.45	0.00	9	10
3	300M	4	299.72	299.72	0.00	4	10	299.72	299.72	0.00	7	10
3	300S	4	251.64	251.64	0.00	2	10	251.64	251.64	0.00	2	10
3	180L	5	400.83	400.83	0.00	33	10	400.83	400.83	0.00	38	10
3	180M	5	328.53	328.53	0.00	26	10	328.53	328.53	0.00	37	10
3	180S	5	261.00	261.00	0.00	6	10	261.00	261.00	0.00	13	10
3	240L	5	372.36	372.36	0.00	14	10	372.36	372.36	0.00	13	10
3	240M	5	298.66	298.66	0.00	9	10	298.66	298.66	0.00	13	10
3	240S	5	251.64	251.64	0.00	4	10	251.64	251.64	0.00	5	10
3	300L	5	372.20	372.20	0.00	13	10	372.20	372.20	0.00	12	10
3	300M	5	298.66	298.66	0.00	9	10	298.66	298.66	0.00	12	10
3	300S	5	251.64	251.64	0.00	4	10	251.64	251.64	0.00	5	10
4	180L	4	491.32	491.32	0.00	167	10	491.32	491.32	0.00	313	10
4	180M	4	424.71	427.00	0.44	683	10	427.00	427.00	0.00	519	10
4	180S	4	334.53	334.53	0.00	118	10	334.53	334.53	0.00	50	10
4	240L	4	445.79	445.79	0.00	85	10	445.79	445.79	0.00	68	10
4	240M	4	374.48	374.48	0.00	63	10	374.48	374.48	0.00	50	10
4	240S	4	322.41	322.41	0.00	51	10	322.41	322.41	0.00	14	10
4	300L	4	444.67	444.67	0.00	88	10	444.67	444.67	0.00	65	10
4	300M	4	374.48	374.48	0.00	67	10	374.48	374.48	0.00	49	10
4	300S	4	322.41	322.41	0.00	44	10	322.41	322.41	0.00	13	10
5	180L	4	537.39	564.10	5.13	1996	5	564.10	564.10	0.00	878	7
5	180M	4	458.14	467.39	1.85	1849	6	457.42	467.39	2.13	976	7
5	180S	4	404.85	404.85	0.00	676	10	404.85	404.85	0.00	123	10
5	240L	4	521.78	528.26	1.30	1265	10	528.26	528.26	0.00	213	10
5	240M	4	443.19	452.04	1.71	1341	10	452.04	452.04	0.00	417	10
5	240S	4	389.88	389.88	0.00	283	10	389.88	389.88	0.00	36	10
5	300L	4	519.60	527.92	1.62	1353	10	526.71	526.71	0.00	173	10
5	300M	4	444.18	449.88	1.02	1252	10	449.88	449.88	0.00	315	10
5	300S	4	389.65	389.65	0.00	322	10	389.65	389.65	0.00	32	10

there is no striking difference. However, as the number of requests increases to 5, solving times grow and the LBB method starts to converge to optimality more quickly. Notably, several instances with 5 requests are not solved to optimality by the MILP, whereas the LBB reaches optimality for nearly all but the instance groups where neither method could find feasible solutions for all instances. These results suggest that the LBB is more scalable, making it a strong candidate for solving larger instances efficiently. Table 11 in the Appendix, presents the detailed results.

5.2 Large instances

Generator of benchmarks. To evaluate the performance of our approach, we develop an instance generator for PDPT. Each generated instance is defined by three sets: a set of transportation requests $\mathcal{R}$, a fleet of vehicles $\mathcal{K}$, and a set of transfer points $\mathcal{T}$. A request r is specified by a pickup location p_r , a delivery

location d_r , and a cargo quantity q_r to be transported between the two. Each vehicle k is defined by its origin o_k and destination locations e_k , along with a maximum cargo capacity Q . Transfer points serve as intermediate transfer locations and are defined solely by their geographic coordinates.

Each location in the instance (those associated with requests, vehicles, and transfer points) is characterised by a set of geographical coordinates, a service duration, and a time window indicating the allowable time frame for the start of service. Travel times between locations are computed using the haversine distance (in hecatometers) divided by a fixed vehicle speed, yielding the time in minutes. Regarding the geographical coordinates of the locations, we utilise the street map of Athens, Greece, as provided by OpenStreetMap¹. We retain only the nodes located within a predefined radius from the city’s center.

The generation of requests follows a feasibility-first approach. Each request is sampled by first proposing pickup and delivery locations, along with associated service durations and time windows. A request is accepted only if there exists a feasible direct route for a single vehicle to depart from its origin location, visit the pickup location and perform the required service, proceed to the delivery location and complete the delivery, and terminate at its destination location. This route must satisfy all relevant time window constraints. If the request passes this check, the cargo quantity is sampled from a predefined distribution, and the request is added to the instance. This process is repeated until $|\mathcal{R}|$ valid requests are generated.

To determine the number of vehicles available in the instance, we consider the same problem but without transfers (PDP). We perform a binary search over the number of vehicles to find the smallest fleet size for which a feasible solution can be found through a simple cheapest-insertion heuristic. Finally, we generate transfer locations by performing k-Means clustering on the set of all geographic coordinates associated with the generated requests. The centroids of the resulting clusters are used as the coordinates of the transfer locations.

Generated instances. Using the generator described above, we produce a diverse collection of benchmark instances designed to reflect realistic operational settings and varying levels of complexity. The load q_r associated with request r is u.a.r. $[5, 25]$. The vehicle capacity Q is set to five times the average cargo quantity, resulting in $Q = 75$, to ensure that vehicles can serve multiple requests. All time-related quantities are modelled assuming an 8-hour workday, beginning at time 0 and ending at time 480 (minutes). Vehicle travel speed is fixed at 20 km/h. Time is measured in minutes, and travel times are computed by converting haversine distances (in hecatometers) into durations based on this speed.

To generate realistic time windows, the workday is partitioned into 16 half-hour intervals. For each location, the start of its time window is drawn uniformly from the set $0, 30, 60, \dots, 450$. The service duration required at each location is u.a.r. $[3, 10]$ minutes. Three levels of time window width were considered: small ‘S’ (60 or 90 minutes), medium ‘M’ (90 or 120 minutes) and large ‘L’ (120 or 150 minutes). Each generated instance uses a fixed time window width selected from one of the above ranges.

We generate instances at four different scales, defined by the number of requests and transfer points: 25 requests and 3 transfer points, 50 requests with 4 transfer points, 75 requests with 5 transfer points and 100 requests with 6 transfer points. For each scale and time-window width (small, medium, large), we produce five instances, resulting in a total of 60 instances. This systematic variation enables a thorough analysis of algorithmic performance across diverse problem sizes and time constraints. A summary of the generator’s parameters is displayed in Table 5. The code of the generator and the instances are publicly available.²

¹<https://www.openstreetmap.org/>

²https://github.com/tsompanidisn/PDPTW_T_BENCH

Table 5: Summary of generator parameters

Parameter	Uniform
$ \mathcal{R} $	25, 50, 75, 100
q_r	integer[5, 25]
Q	75
l_j	0, 30, 60, $\dots$, 450 for demand locations, 0 for depots and transfer points
u_j	$l_j +$: 60 or 90 for ‘S’, 90 or 120 for ‘M’, 120 or 150 for ‘L’ - set to 480 for depots and transfer points
$st(j)$	integer[3, 10]
$ \mathcal{T} $	3 for 25 requests, 4 for 50 requests, 5 for 75 requests, 6 for 100 requests

Performance of Branch-and-Check. LBBD is applied to the generated benchmarks of tractable scale for the master problem, i.e., with 25 and 50 requests. Initially, the algorithm is tested as a standalone method. However, due to its difficulty in retrieving a feasible solution independently, a second set of experiments is conducted, where the solution from the refined LNS algorithm is used as a warm-start for the master problem.

Table 6 presents the results for both experimental setups. The column ‘LBBD’ reports the performance of the exact method without warm-start support, while ‘LBBD⁺’ shows the results after incorporating the LNS solution into the master problem. Additionally, the ‘rLNS’ column provides the objective value of the solution obtained by our LNS method. No time results are given as LBBD reaches the time limit in all instances (with or without warm-start).

Table 6: Results of the Branch-and-Check on the generated benchmarks

Requests	Vehicles	Transfer points	Time windows	Variant	LBBD			rLNS	LBBD ⁺		
					LB	UB	Gap	UB	LB	UB	Gap
25	3	3	L	1	992.36	-	-	1664	993.43	1664	40.30
	3	3	L	2	887.94	-	-	1590	889.32	1423	37.50
	3	3	L	3	926.17	-	-	1585	953.96	1585	39.81
	4	3	L	4	856.43	-	-	1556	860.68	1526	43.60
	3	3	L	5	902.16	1610	43.97	1404	904.86	1389	34.86
	4	3	M	1	1069.95	-	-	1534	1070.77	1506	28.90
	3	3	M	2	1034.24	-	-	1674	1044.23	1674	37.62
	4	3	M	3	991.66	-	-	1584	1001.00	1584	36.81
	4	3	M	4	899.36	-	-	1445	900.21	1441	37.53
	4	3	M	5	1013.30	1781	43.10	1855	1034.71	1717	39.74
	4	3	S	1	1058.82	1786	40.72	1754	1082.88	1754	38.26
	4	3	S	2	1009.01	-	-	1646	1015.89	1646	38.28
	4	3	S	3	1082.40	-	-	1649	1090.70	1649	33.86
	4	3	S	4	1177.44	-	-	1682	1187.45	1647	27.90
50	3	3	S	5	1123.87	-	-	1757	1149.31	1757	34.59
	6	4	L	1	1441.58	-	-	2706	1442.01	2706	46.71
	6	4	L	2	1370.09	-	-	2642	1379.90	2642	47.77
	5	4	L	3	1297.64	-	-	2733	1298.51	2733	52.49
	7	4	L	4	1363.51	-	-	2632	1369.77	2632	47.96
	6	4	L	5	1357.78	-	-	2604	1359.99	2604	47.77
	7	4	M	1	1550.29	-	-	2614	1562.00	2614	40.24
	7	4	M	2	1472.47	-	-	2942	1482.12	2942	49.62
	7	4	M	3	1543.21	-	-	2812	1550.96	2807	44.75
	7	4	M	4	1493.33	-	-	2933	1493.58	2914	48.74
	7	4	M	5	1422.21	-	-	2821	1424.77	2821	49.49
	7	4	S	1	1773.74	-	-	3461	1786.71	3393	47.34
	6	4	S	2	1765.88	-	-	3199	1780.09	3171	43.86
	7	4	S	3	1583.91	-	-	2852	1586.11	2737	42.05
	7	4	S	4	1685.68	-	-	3196	1707.29	3196	46.58
	7	4	S	5	1625.96	-	-	2947	1638.86	2905	43.58

The results indicate that the LBB algorithm struggles to produce feasible solutions, as an upper bound (‘UB’) is reported for only 3 out of 30 instances, all of which involve 25 requests. When a warm-start solution is provided, a slight improvement in the lower bound (‘LB’) is observed across all 30 instances. In almost half of the cases (13 out of 30), the LNS solution is improved, yielding a reduced upper bound. The corresponding instances are highlighted in bold.

Performance of rLNS. For larger datasets, we apply the LNS algorithm described in Section 4. To assess the effectiveness of rLNS, we conduct a series of computational experiments involving two alternative approaches: ‘LS’ and ‘MULTI-OP’. The LS metaheuristic is structurally similar to rLNS, with the primary distinction being the use of the original removal operator of [10], in contrast to the modified version used in rLNS. The second baseline, MULTI-OP, is built on the ALNS approach [25], i.e., it operates by iteratively selecting pairs of destroy and repair operators, guided by an adaptive mechanism that updates selection probabilities based on recent performance. The destroy phase of MULTI-OP employs three removal strategies: random removal and worst removal as defined in [25], as well as the similarity-based removal described in Section 4. For the repair phase, two request insertion strategies were examined: one involving a random request ordering, and the other using the Insertion Difficulty criterion, as introduced in Section 4.

The three methods have been tested on a diverse set of problem instances tailored for PDPT, generated by the aforementioned rules at two different scales, namely 75 and 100 requests. Table 7 summarises the hyperparameter configurations used for each of the three methods evaluated in this experiment. Shared parameters, such as the LAHC [5] list size and degree of destruction, are kept consistent to ensure a fair comparison, while method-specific parameters reflect the unique mechanisms of each approach.

Table 7: Hyperparameter settings for each method

Parameter	Description	Value(s)
rLNS		
L	LAHC [5] list size	20
DR	Degree of destruction (uniform range)	[5, 15]
β	Blink rate for repair operator	0.05
LS		
L	LAHC [5] list size	20
DR	Degree of destruction (uniform range)	[5, 15]
β	Blink rate for repair operator	0.05
$\theta_1, \theta_2, \theta_3$	Shaw removal operator coefficients	0.33, 0.99, 0.66
MULTI-OP		
L	LAHC [5] list size	20
DR	Degree of destruction (uniform range)	[5, 15]
β	Blink rate for repair operator	0.05
w_s	Cost scaling factor for worst removal	0.25
α	Learning rate for operator weights	0.3
s_B	Reward for improving best solution	3
s_A	Reward for accepted solution	1

Table 8 presents the comparative results for MULTI-OP, LS and rLNS. To mitigate the effect of the algorithms’ randomness on the experimental results, we follow a multi-restart approach, i.e. each algorithm runs $K = 10$ times on each problem instance. For each restart, the algorithm is terminated when the local search performs $p = 50$ successive iterations without improving the best solution encountered. The ‘Best UB’ columns report the objective value of the best-found solution across all restarts. Columns ‘Avg UB’ indicate the average objective value of the best-found solution per restart, and columns ‘Time’ indicate the average execution time per restart in seconds.

In columns ‘Best UB’ and ‘Avg UB’, the numbers in bold indicate that the corresponding value is better than the respective result of the compared methods. As observed, our refined LNS algorithm improves the

Table 8: Comparison between MULTI-OP, LS and the rLNS

$\mathcal{R}$	TW	Variant	MULTI-OP			LS			rLNS		
			Best UB	Avg UB	Time	Best UB	Avg UB	Time	Best UB	Avg UB	Time
75	L	1	3689	4038	36	3767	4051	33	3644	3866	41
	L	2	3429	3711	46	3590	3918	29	3410	3881	32
	L	3	3608	4071	32	3946	4268	23	3683	3904	35
	L	4	3388	3689	70	3567	3845	37	3352	3676	42
	L	5	3532	3894	45	3582	3926	35	3403	3751	49
	M	1	3548	3686	61	3604	3849	37	3478	3685	66
	M	2	3382	3690	52	3203	3798	40	3350	3652	37
	M	3	3642	3792	55	3490	3821	47	3498	3739	68
	M	4	3516	3756	45	3588	3760	39	3467	3651	54
	M	5	3718	3969	46	3573	4037	40	3584	3804	57
	S	1	4064	4328	39	4049	4222	42	3917	4194	37
	S	2	4025	4331	41	4042	4459	35	3883	4202	43
	S	3	3983	4210	56	4259	4573	27	4004	4287	37
	S	4	4129	4342	38	4091	4546	24	4004	4254	39
	S	5	3930	4223	45	3828	4343	36	3778	4044	48
100	L	1	4596	4787	152	4730	4942	158	4667	4877	175
	L	2	4283	4875	87	4348	4757	148	4232	4487	215
	L	3	4614	4863	188	4500	4773	227	4535	4904	207
	L	4	4195	4698	119	4357	4682	192	4145	4570	165
	L	5	4560	4822	145	4677	4905	187	4702	5173	95
	M	1	4738	5054	117	4773	5100	154	4629	4987	198
	M	2	4835	5140	129	4756	5079	201	4748	4989	222
	M	3	5058	5374	96	4981	5240	178	4898	5304	166
	M	4	4827	5174	102	4847	5127	138	4772	5055	171
	M	5	4711	5152	111	4850	5183	150	4869	5319	140
	S	1	4735	5000	149	4874	5086	112	4682	5111	132
	S	2	5139	5552	129	5230	5488	130	5117	5503	128
	S	3	4867	5134	126	5037	5325	107	4894	5356	93
	S	4	4936	5170	157	4969	5221	152	5013	5310	135
	S	5	5106	5294	130	4894	5295	209	5070	5316	167

best-found solution for 18 out of 30 instances - 10 for 75 requests and 8 for 100 requests. Regarding the average objective value of the best-found solutions per restart, rLNS provides the best-of-three value for 18 out of 30 instances - 13 for 75 requests and 5 for 100 requests.

The experimental results highlight the effectiveness and robustness of the proposed refined LNS (rLNS) algorithm for solving large-scale PDPT instances. Its consistent performance across diverse scenarios suggests strong generalisation capabilities, despite its reduced reliance on extensive parameter tuning. This makes rLNS easier to apply and more likely to perform well on unseen problem instances.

6 Concluding remarks

This paper proposes exact and metaheuristic approaches for the Pickup and Delivery Problem with Transfers and Time Windows (PDPT), a highly applicable yet underexplored variant that generalises all classical routing problems. The exact method is driven by a novel partitioning strategy that constructs parallel paths for all pickup and delivery requests, to be then consolidated into a unified routing plan. This idea enables the design of a Logic-Based Benders Decomposition (LBBD) scheme. To address the limited scalability of the exact method, we also propose a Large Neighborhood Search (LNS) algorithm that builds on foundational ideas from related work, resulting in a more generalisable and effective solution approach.

The LBBD scheme, implemented as a Branch-and-Check algorithm, demonstrates significant improvements over state-of-the-art exact methods in the literature for the same problem. Experimental results on existing benchmarks show either faster convergence to optimality or reduced optimality gaps for instances

with up to 30 requests, i.e., the largest available in the current literature. The performance of the LBBB approach is further enhanced when the solution from the proposed LNS algorithm is used as a warm-start.

Due to the lack of larger benchmarks, we also propose a random instance generator that incorporates more realistic assumptions. Existing works often assume large-sized requests that severely limits the number of requests accommodated in a vehicle. In contrast, our generator allows for more practical scenarios where up to 15 requests fit in a single vehicle. For these generated instances, we use the exact method to compute lower bounds, enabling a meaningful evaluation of the solutions produced by our LNS. Subsequently, we compare our LNS against state-of-the-art metaheuristic algorithms, thus showing that our method yields improved best-found solutions in most tested instances.

Our findings could motivate further effort in multiple directions. One may focus on more sophisticated conflict analysis, performed at solutions that lead to infeasible subproblems, to identify infeasible sub-routes; this might generate stronger feasibility cuts, further enhancing the efficiency of the method. Additionally, a hybrid matheuristic approach could be explored, where the master problem serves as a neighborhood constructor, while the LNS algorithm acts as a fast, adaptive mechanism to explore the defined neighborhood and identify improved solutions.

A key limitation in scaling current metaheuristic approaches for the PDPT lies in the computational cost of the ‘request insertion’ operation, which increases significantly with the number of requests. One potential way to address this challenge is through decomposition techniques - such as those explored in [34] for classical vehicle routing problems - which divide the original problem into smaller, more manageable subproblems. However, applying such techniques to the PDPT is not straightforward due to the interdependence between vehicles introduced by the use of transshipment points. Designing a decomposition strategy that accounts for these interactions presents a non-trivial challenge and offers an interesting direction for future research aimed at improving scalability.

Several routing problems are reducible to PDPT and hence eligible for applying our approach. An indicative example is the VRP with Transfers having only pickups or only deliveries. Specific problems involving mobile depots that facilitate load transfers also fall within the scope of applicability. Beyond routing, synchronisation constraints similar to those in the PDPT frequently arise in scheduling problems, hence the decomposition strategies and metaheuristic techniques presented in this paper may offer valuable insights.

Acknowledgments This research has been supported by GREEN-LOG Horizon Europe Project [grant agreement 101069892].

References

- [1] Accorsi, L., Vigo, D., Routing one million customers in a handful of minutes. *Computers & Operations Research*, 164, 106562 (2024).
- [2] Aguayo, M.M., Avilés, F.N., Sarin, S.C., Archetti, C., The vehicle routing problem with transfers. *Computers & Operations Research*, 177, 106980 (2025).
- [3] Baldacci, R., Ulrich Nguereu, S., Calvo, R.W., The Vehicle Routing Problem with Transshipment Facilities. *Transportation Science*, 51:2, 592–606 (2017).
- [4] Benders J.F., Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4, 238-252 (1962).

- [5] Burke, E.K., Bykov, Y., The late acceptance Hill-Climbing heuristic. *European Journal of Operational Research*, 258:1, 70-78 (2017).
- [6] Christiaens, J., Vanden Berghe, G., Slack induction by string removals for vehicle routing problems. *Transportation Science*, 54:2, 417-433 (2020).
- [7] Codato, G., Fischetti, M., Combinatorial Benders' Cuts for Mixed-Integer Linear Programming. *Operations Research*, 54:4, 756-766 (2006).
- [8] Cordeau, J.F., Laporte, G., The dial-a-ride problem: models and algorithms. *Annals of Operations Research*, 153, 29-46 (2007).
- [9] Cortés, C.E., Matamala, M., Contardo, C., The pickup and delivery problem with transfers: Formulation and a branch-and-cut solution method. *European Journal of Operational Research*, 200, 711-724 (2010).
- [10] Danloup, N., Allaoui, H., Goncalves, G., A comparison of two meta-heuristics for the pickup and delivery problem with transshipment. *Computers & Operations Research*, 100, 155-171 (2018).
- [11] Drexl, M., Applications of the vehicle routing problem with trailers and transshipments. *European Journal of Operational Research*, 227, 275-283 (2013).
- [12] Dumas, Y., Desrosiers, J., Soumis, F., The pickup and delivery problem with time windows. *European Journal of Operational Research*, 54, 7-22 (1991).
- [13] Hof, J., Schneider, M., Intraroute Resource Replenishment with Mobile Depots. *Transportation Science*, 55:3, 660-686 (2021).
- [14] Hooker, J.N., Ottoson, G., Logic-Based Benders Decomposition. *Mathematical Programming*, 96, 33-60 (2003).
- [15] Kirkpatrick, S., Gelatt, Jr.C.D., Vecchi, M.P., Optimization by simulated annealing. *Science*, 220, 671-680 (1983).
- [16] Koç, Ç., Laporte, G., Tükenmez, İ., A review of vehicle routing with simultaneous pickup and delivery. *Computers & Operations Research*, 122, 104987 (2020).
- [17] Lyu, Z., Junfang Yu, A., The pickup and delivery problem with transshipments: Critical review of two existing models and a new formulation. *European Journal of Operational Research*, 305, 260-270 (2023).
- [18] Masson, R., Lehuédé, F., Péton, O., Efficient feasibility testing for request insertion in the pickup and delivery problem with transfers. *Operations Research Letters*, 41:3, 211-215 (2013).
- [19] Masson, R., Lehuédé, F., Péton, O., An Adaptive Large Neighborhood Search for the Pickup and Delivery Problem with Transfers. *Transportation Science*, 47:3, 44-355 (2013).
- [20] Masson, R., Ropke, S., Lehuédé, F., Péton, O., A branch-and-cut-and-price approach for the pickup and delivery problem with shuttle routes. *European Journal of Operational Research*, 236, 849-862 (2014).
- [21] Mitrović-Minić, S., Laporte, G., The Pickup And Delivery Problem With Time Windows And Transshipment. *INFOR: Information Systems and Operational Research*, 44:3, 217-227 (2006).
- [22] Pisinger, D., Ropke, S., Large Neighbourhood Search. *Handbook of Metaheuristics, International Series in Operations Research & Management Science*, (2018).

- [23] Qu, Y., Bard, J.F., A GRASP with adaptive large neighborhood search for pickup and delivery problems with transshipment. *Computers & Operations Research*, 39:10, 2439-2456 (2012).
- [24] Rais, A., Alvelos, F., Carvalho, M.S., New mixed integer-programming model for the pickup-and-delivery problem with transshipment. *European Journal of Operational Research*, 235, 530-539 (2014).
- [25] Ropke, S., Pisinger, D., An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40:4, 455-472 (2006).
- [26] Shaw, P., Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems. *Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems, CP1998*, 417-431 (1998).
- [27] Sampaio, A., Savelsbergh, M., Veelenturf, L.P., Van Woensel, T., Delivery systems with crowd-sourced drivers: A pickup and delivery problem with transfers. *Networks*, 76, 232-255 (2020).
- [28] Sartori, C.S., Buriol, L.S., A study on the pickup and delivery problem with time windows: Matheuristics and new instances. *Computers & Operations Research*, 124, 105065 (2020).
- [29] Savelsbergh, M.W.P., Sol, M., The General Pickup and Delivery Problem. *Transportation Science*, 29:1, 17-29 (1995).
- [30] Thorsteinsson, E.S., Branch-and-Check: A Hybrid Framework Integrating Mixed Integer Programming and Constraint Logic Programming. *CP 2001: Principles and Practice of Constraint Programming*, 16-30 (2001).
- [31] Toth, P., Vigo, D., The Vehicle Routing Problem. *Society for Industrial and Applied Mathematics*, (2002).
- [32] Voigt, S., Kuhn, H., Crowdsourced logistics: The pickup and delivery problem *Networks*, 79:3, 403-426 (2022).
- [33] Zhang, Q., Wang, Z., Huang, M., Yu, Y., Fang, S.C., Heterogeneous multi-depot collaborative vehicle routing problem. *Transportation Research Part B: Methodological*, 160, 1-20 (2022).
- [34] Alberto Santini, Michael Schneider, Thibaut Vidal, Daniele Vigo, Decomposition Strategies for Vehicle Routing Heuristics. *INFORMS Journal on Computing* 35(3):543-55 (2023).

Appendix

Table 9: Exact methods on instances of [17] - loose time windows

$\mathcal{R}$	$\mathcal{K}$	$\mathcal{T}$	Variant	MILP [17]				LBBD			
				LB	UB	Gap (%)	Time (s)	LB	UB	Gap (%)	Time (s)
10	2	1	0	649.36	649.36	0.00	2	649.36	649.36	0.00	5
			1	716.40	716.40	0.00	3	716.40	716.40	0.00	2
			2	660.26	660.26	0.00	22	660.26	660.26	0.00	3
			3	673.13	673.13	0.00	12	673.13	673.13	0.00	6
			4	821.16	821.16	0.00	5	821.16	821.16	0.00	4
			5	653.01	653.01	0.00	1	653.01	653.01	0.00	0
			6	782.91	782.91	0.00	49	782.91	782.91	0.00	4
			7	765.55	765.55	0.00	7	765.55	765.55	0.00	2
			8	619.03	619.03	0.00	6	619.03	619.03	0.00	5
			9	831.86	831.86	0.00	1	831.86	831.86	0.00	2
		2	0	798.72	798.72	0.00	8	798.72	798.72	0.00	19
			1	799.68	799.68	0.00	25	799.68	799.68	0.00	6
			2	753.83	753.83	0.00	4	753.83	753.83	0.00	2
			3	677.08	677.08	0.00	6	677.08	677.08	0.00	9
			4	675.45	675.45	0.00	26	675.45	675.45	0.00	13
			5	614.27	614.27	0.00	0	614.27	614.27	0.00	2
			6	627.97	627.97	0.00	115	627.97	627.97	0.00	42
			7	652.52	652.52	0.00	17	652.52	652.52	0.00	2
			8	708.10	708.10	0.00	0	708.10	708.10	0.00	0
			9	722.76	722.76	0.00	9	722.76	722.76	0.00	3
	3	3	0	665.42	665.42	0.00	1	665.42	665.42	0.00	9
			1	665.32	665.32	0.00	12	665.32	665.32	0.00	12
			2	667.10	667.10	0.00	24	667.10	667.10	0.00	9
			3	686.70	686.70	0.00	7	686.70	686.70	0.00	44
			4	684.53	684.53	0.00	2	684.53	684.53	0.00	29
			5	783.94	783.94	0.00	3	783.94	783.94	0.00	17
			6	662.78	662.78	0.00	3	662.78	662.78	0.00	7
			7	671.41	671.41	0.00	19	671.41	671.41	0.00	123
			8	764.94	764.94	0.00	1	764.94	764.94	0.00	1
12	2	1	0	653.10	653.10	0.00	56	653.10	653.1	0.00	8
			1	718.52	718.52	0.00	356	718.52	718.52	0.00	12
			2	862.64	862.64	0.00	1	862.64	862.64	0.00	0
			3	684.59	684.59	0.00	3	684.59	684.59	0.00	1
			4	815.49	815.49	0.00	3	815.49	815.49	0.00	4
			5	708.07	708.07	0.00	1	708.07	708.07	0.00	0
			6	780.58	780.58	0.00	2	780.58	780.58	0.00	1
			7	690.09	690.09	0.00	1	690.09	690.09	0.00	2
			8	767.34	767.34	0.00	2	767.34	767.34	0.00	2
			9	610.22	610.22	0.00	6	610.22	610.22	0.00	2
		2	0	802.56	802.56	0.00	1	802.56	802.56	0.00	0
			1	849.74	849.74	0.00	139	849.74	849.74	0.00	5
			2	601.69	601.69	0.00	239	601.69	601.69	0.00	45
			3	1107.36	1107.36	0.00	2	1107.36	1107.36	0.00	1
			4	595.99	595.99	0.00	63	595.99	595.99	0.00	12
			5	688.88	688.88	0.00	13	688.88	688.88	0.00	6
			6	679.54	679.54	0.00	89	679.54	679.54	0.00	7
			7	814.21	814.21	0.00	91	814.21	814.21	0.00	8
			8	710.00	710.00	0.00	59	710.00	710.00	0.00	4
			9	666.03	666.03	0.00	20	666.03	666.03	0.00	22
	3	3	0	727.75	727.75	0.00	643	727.75	727.75	0.00	33
			1	888.17	888.17	0.00	165	888.17	888.17	0.00	18
			2	932.85	932.85	0.00	20	932.85	932.85	0.00	8
			3	827.47	827.47	0.00	110	827.47	827.47	0.00	7
			4	708.61	708.61	0.00	20	708.61	708.61	0.00	22
			5	883.58	883.58	0.00	43	883.58	883.58	0.00	1242
			6	778.26	778.26	0.00	2	778.26	778.26	0.00	1
			7	946.76	946.76	0.00	1052	946.76	946.76	0.00	7
			8	752.98	752.98	0.00	36	752.98	752.98	0.00	26
			9	677.20	677.20	0.00	86	677.20	677.2	0.00	178
			0	1115.03	1115.03	0.00	8	1115.03	1115.03	0.00	4
			1	1031.72	1031.72	0.00	700	1031.72	1031.72	0.00	6

			2	1075.41	1075.41	0.00	11	1075.41	1075.41	0.00	1
			3	933.07	933.07	0.00	8	933.07	933.07	0.00	6
			4	806.86	806.86	0.00	190	806.86	806.86	0.00	10
			5	873.79	873.79	0.00	758	873.79	873.79	0.00	19
			6	824.24	824.24	0.00	90	824.24	824.24	0.00	8
			7	884.41	884.41	0.00	5	884.41	884.41	0.00	2
			8	1013.17	1013.17	0.00	1787	1013.17	1013.17	0.00	62
			9	772.37	772.37	0.00	59	772.37	772.37	0.00	12
			0	891.36	891.36	0.00	11	891.36	891.36	0.00	83
		2	1	950.85	950.85	0.00	3	950.85	950.85	0.00	1
			2	1104.09	1104.09	0.00	19	1104.09	1104.09	0.00	5
			3	985.43	985.43	0.00	144	985.43	985.43	0.00	9
			4	655.07	655.07	0.00	642	655.07	655.07	0.00	20
			5	1003.45	1003.45	0.00	8	1003.45	1003.45	0.00	25
			6	1018.54	1018.54	0.00	31	1018.54	1018.54	0.00	9
			7	891.25	891.25	0.00	73	891.25	891.25	0.00	11
			8	937.02	937.02	0.00	77	937.02	937.02	0.00	64
			9	799.54	799.54	0.00	73	799.54	799.54	0.00	8
			0	895.40	895.40	0.00	89	895.40	895.40	0.00	494
			1	926.87	926.87	0.00	80	926.87	926.87	0.00	21
			2	834.61	834.61	0.00	707	834.61	834.61	0.00	80
			3	1070.11	1070.11	0.00	54	1070.11	1070.11	0.00	29
			4	1159.83	1159.83	0.00	33	1159.83	1159.83	0.00	8
			5	966.15	966.15	0.00	20	966.15	966.15	0.00	56
			6	960.89	960.89	0.00	211	960.89	960.89	0.00	11
			7	888.21	888.21	0.00	1070	888.21	888.21	0.00	55
			8	877.48	877.48	0.00	26	877.48	877.48	0.00	3411
			9	884.22	901.26	1.89	3600	901.26	901.26	0.00	259
			0	938.50	976.51	3.89	3600	973.38	973.38	0.00	392
			1	1295.13	1295.13	0.00	2094	1295.13	1295.13	0.00	189
			2	985.27	1012.07	2.65	3600	1012.07	1012.07	0.00	159
			3	859.79	1004.95	14.44	3600	945.96	945.96	0.00	369
			4	1403.14	1403.14	0.00	1208	1403.14	1403.14	0.00	40
			5	1162.67	1162.67	0.00	55	1162.67	1162.67	0.00	12
			6	1251.04	1251.04	0.00	39	1251.04	1251.04	0.00	37
			7	-	-	-	3600	1177.13	1177.13	0.00	498
			8	1365.88	1365.88	0.00	65	1365.88	1365.88	0.00	5
			9	958.51	1001.81	4.32	3600	983.92	983.92	0.00	86
			0	-	-	-	3600	1068.95	1165.24	8.26	3600
			1	1351.70	1351.7	0.00	1970	1351.70	1351.7	0.00	917
			2	1121.48	1166.65	3.87	3600	1160.81	1160.81	0.00	364
			3	-	-	-	3600	949.35	976.16	2.75	3600
			4	1357.37	1357.37	0.00	3171	1357.37	1357.37	0.00	53
			5	1067.78	1090.4	2.07	3600	1090.40	1090.4	0.00	187
			6	984.65	984.65	0.00	1182	984.65	984.65	0.00	92
			7	-	-	-	3600	1099.81	1099.81	0.00	721
			8	-	-	-	3600	1118.42	1118.42	0.00	145
			9	982.45	982.45	0.00	230	982.45	982.45	0.00	11
			0	1186.43	1186.43	0.00	285	1186.43	1186.43	0.00	36
			1	969.31	1044.14	7.17	3600	986.76	997.42	1.07	3600
			2	999.90	1444.48	30.78	3600	1060.14	1060.14	0.00	2488
			3	1069.09	1069.09	0.00	405	1069.09	1069.09	0.00	19
			4	1356.11	1356.11	0.00	1250	1359.17	1359.17	0.00	22
			5	936.63	1135.73	17.53	3600	1007.22	1007.22	0.00	905
			6	-	-	-	3600	1011.58	1026.64	1.47	3600
			7	1124.50	1198.01	6.14	3600	1146.66	1146.66	0.00	186
			8	1097.71	1158.17	5.22	3600	1143.91	1143.91	0.00	164
			9	1012.56	1082.21	6.44	3600	1060.92	1060.92	0.00	1021
			0	-	-	-	3600	1428.23	1450.07	1.51	3600
			1	-	-	-	3600	1305.34	1428.17	8.60	3600
			2	-	-	-	3600	816.36	1006.54	18.89	3600
			3	1523.79	1523.79	0.00	1185	1523.79	1523.79	0.00	15
			4	-	-	-	3600	1106.58	1130.62	2.13	3600
			5	-	-	-	3600	1547.23	1547.23	0.00	64
			6	-	-	-	3600	1292.03	1292.03	0.00	188
			7	-	-	-	3600	1066.44	1085.33	1.74	3600
			8	-	-	-	3600	1523.96	1523.96	0.00	190
			9	1239.83	1355.21	8.51	3600	1309.90	1309.9	0.00	65

30	2	2	0	1130.04	1361.2	16.98	3600	1186.81	1229.93	3.51	3600
			1	-	-	-	3600	1274.16	1274.16	0.00	621
			2	1429.65	1429.65	0.00	3244	1429.65	1429.65	0.00	279
			3	-	-	-	3600	1350.47	1375.78	1.84	3600
			4	-	-	-	3600	1109.15	1109.15	0.00	2709
			5	-	-	-	3600	1352.49	1352.49	0.00	518
			6	-	-	-	3600	1183.44	1199.72	1.36	3600
			7	-	-	-	3600	1350.78	1371.16	1.49	3600
			8	-	-	-	3600	1243.33	1345.49	7.59	3600
			9	-	-	-	3600	1413.98	1434.38	1.42	3600
		3	0	-	-	-	3600	1268.09	1331.85	4.79	3600
			1	-	-	-	3600	1287.89	1287.89	0.00	456
			2	-	-	-	3600	1302.28	1302.28	0.00	291
			3	1482.03	1525.25	2.83	3600	1517.91	1517.91	0.00	109
			4	-	-	-	3600	1204.83	1204.83	0.00	116
			5	-	-	-	3600	1094.72	1174.37	6.78	3600
			6	1246.98	1529.83	18.49	3600	1302.50	1302.5	0.00	1354
			7	-	-	-	3600	1506.77	1506.77	0.00	706
			8	-	-	-	3600	1316.79	1330.5	1.03	3600
			9	1283.36	1464.54	12.37	3600	1308.66	1308.66	0.00	868
	2	1	0	-	-	-	3600	1440.08	1558.12	7.58	3600
			1	-	-	-	3600	1368.64	1523.67	10.17	3600
			2	-	-	-	3600	1892.81	1892.81	0.00	2879
			3	-	-	-	3600	1512.76	1537.66	1.62	3600
			4	-	-	-	3600	1411.69	1612.99	12.48	3600
			5	-	-	-	3600	1489.06	1572.5	5.31	3600
			6	-	-	-	3600	1826.05	1826.05	0.00	563
			7	-	-	-	3600	1456.76	1488.13	2.11	3600
			8	-	-	-	3600	1489.68	1489.68	0.00	1211
			9	1412.27	1574.83	10.32	3600	1454.08	1463.82	0.67	3600
		2	0	-	-	-	3600	1562.80	1628.16	4.01	3600
			1	-	-	-	3600	-	-	-	3600
			2	-	-	-	3600	1511.09	1648.79	8.35	3600
			3	-	-	-	3600	1224.25	1341.17	8.72	3600
			4	-	-	-	3600	-	-	-	3600
			5	-	-	-	3600	-	-	-	3600
			6	-	-	-	3600	1540.55	1558.31	1.14	3600
			7	-	-	-	3600	1602.73	1602.73	0.00	2035
			8	-	-	-	3600	1332.66	1397.26	4.62	3600
			9	-	-	-	3600	1201.14	1382.98	13.15	3600
		3	0	-	-	-	3600	1441.28	1448.36	0.49	3600
			1	-	-	-	3600	1149.70	1423.21	19.22	3600
			2	-	-	-	3600	1435.63	1573.89	8.78	3600
			3	-	-	-	3600	-	-	-	3600
			4	-	-	-	3600	1957.16	1957.16	0.00	657
			5	-	-	-	3600	1658.42	1681.16	1.35	3600
			6	-	-	-	3600	1607.70	1607.7	0.00	638
			7	-	-	-	3600	1234.90	1338.7	7.75	3600
			8	-	-	-	3600	1469.13	1475.74	0.45	3600
			9	-	-	-	3600	1359.42	1540.18	11.74	3600

Table 10: LBB and LNS on instances of [17] - loose time windows

$\mathcal{R}$	$\mathcal{K}$	$\mathcal{T}$	Variant	LBB				LNS				LBB ⁺			
				LB	UB	Gap	Time	UB	Err	Gap	Time	LB	UB	Gap	Time
20	2	1	0	973.38	973.38	0.00	392	999.38	2.60	2.60	1	973.38	973.38	0.00	228
			1	1295.13	1295.13	0.00	189	1409.12	8.09	8.09	0	1295.13	1295.13	0.00	127
			2	1012.07	1012.07	0.00	159	1048.37	3.46	3.46	1	1012.07	1012.07	0.00	84
			3	945.96	945.96	0.00	369	993.72	4.81	4.81	1	945.96	945.96	0.00	312
			4	1403.14	1403.14	0.00	40	1422.30	1.35	1.35	1	1403.14	1403.14	0.00	32
			5	1162.67	1162.67	0.00	12	1180.75	1.53	1.53	0	1162.67	1162.67	0.00	9
			6	1251.04	1251.04	0.00	37	1309.69	4.48	4.48	0	1251.04	1251.04	0.00	28
			7	1177.13	1177.13	0.00	498	1192.29	1.27	1.27	0	1177.13	1177.13	0.00	393
			8	1365.88	1365.88	0.00	5	1413.59	3.38	3.38	0	1365.88	1365.88	0.00	4
			9	983.92	983.92	0.00	86	1022.03	3.73	3.73	1	983.92	983.92	0.00	75
		2	0	1068.95	1165.24	8.26	3600	1177.87	1.07	9.04	1	1071.37	1160.81	7.70	3600
			1	1351.70	1351.70	0.00	917	1424.27	5.10	5.10	1	1351.70	1351.70	0.00	790
			2	1160.81	1160.81	0.00	364	1205.95	3.74	3.74	1	1160.81	1160.81	0.00	203
			3	949.35	976.16	2.75	3600	996.89	2.08	4.35	1	953.48	976.16	2.32	3600
			4	1357.37	1357.37	0.00	53	1393.82	2.62	2.62	1	1357.37	1357.37	0.00	41
			5	1090.40	1090.40	0.00	187	1133.87	3.83	3.83	1	1090.40	1090.40	0.00	153
			6	984.65	984.65	0.00	92	1048.44	6.08	6.08	1	984.65	984.65	0.00	79

25	3	3	7	1099.81	1099.81	0.00	721	1131.64	2.81	2.81	1	1099.81	1099.81	0.00	468
			8	1118.42	1118.42	0.00	145	1209.05	7.50	7.50	1	1118.42	1118.42	0.00	128
			9	982.45	982.45	0.00	11	1015.41	3.25	3.25	1	982.45	982.45	0.00	9
			0	1186.43	1186.43	0.00	36	1219.30	2.70	2.70	1	1186.43	1186.43	0.00	31
			1	986.76	997.42	1.07	3600	1042.88	4.36	5.25	1	988.18	997.42	0.93	3600
			2	1060.14	1060.14	0.00	2488	1123.38	5.63	5.63	1	1060.14	1060.14	0.00	2211
			3	1069.09	1069.09	0.00	19	1155.90	7.51	7.51	1	1069.09	1069.09	0.00	15
			4	1359.17	1359.17	0.00	22	1382.77	1.71	1.71	1	1359.17	1359.17	0.00	17
			5	1007.22	1007.22	0.00	905	1051.24	4.19	4.19	1	1007.22	1007.22	0.00	787
			6	1011.58	1026.64	1.47	3600	1061.92	3.32	4.70	1	1011.96	1026.64	1.43	3600
			7	1146.66	1146.66	0.00	186	1242.73	7.73	7.73	1	1146.66	1146.66	0.00	147
			8	1143.91	1143.91	0.00	164	1221.20	6.33	6.33	1	1143.91	1143.91	0.00	123
			9	1060.92	1060.92	0.00	1021	1067.06	0.58	0.58	1	1060.92	1060.92	0.00	904
	1	1	0	1428.23	1450.07	1.51	3600	1518.26	4.49	5.90	1	1428.61	1439.64	0.77	3600
			1	1305.34	1428.17	8.60	3600	1460.31	2.20	10.53	1	1306.52	1398.17	6.55	3600
			2	816.36	1006.54	18.89	3600	979.12	-2.80	10.83	1	873.11	979.12	10.83	3600
			3	1523.79	1523.79	0.00	15	1572.69	3.11	3.11	1	1523.79	1523.79	0.00	12
			4	1106.58	1130.62	2.13	3600	1239.46	8.78	10.66	1	1107.30	1129.27	1.95	3600
			5	1547.23	1547.23	0.00	64	1647.22	6.07	6.07	0	1547.23	1547.23	0.00	50
			6	1292.03	1292.03	0.00	188	1413.64	8.60	8.60	1	1292.03	1292.03	0.00	161
			7	1066.44	1085.33	1.74	3600	1111.95	2.39	4.02	1	1067.26	1080.47	1.22	3600
			8	1523.96	1523.96	0.00	190	1585.77	3.90	3.90	1	1523.96	1523.96	0.00	108
	2	2	9	1309.90	1309.90	0.00	65	1342.99	2.46	2.46	1	1309.90	1309.90	0.00	53
			0	1186.81	1229.93	3.51	3600	1286.84	4.42	6.90	1	1198.02	1229.93	2.59	3600
			1	1274.16	1274.16	0.00	621	1332.58	4.38	4.38	1	1274.16	1274.16	0.00	508
			2	1429.65	1429.65	0.00	279	1458.24	1.96	1.96	1	1429.65	1429.65	0.00	92
			3	1350.47	1375.78	1.84	3600	1458.50	5.67	6.67	1	1361.16	1375.78	1.06	3600
			4	1109.15	1109.15	0.00	2709	1160.51	4.43	4.43	1	1109.15	1109.15	0.00	1669
			5	1352.49	1352.49	0.00	518	1383.94	2.27	2.27	1	1352.49	1352.49	0.00	416
			6	1183.44	1199.72	1.36	3600	1253.66	4.30	5.50	1	1184.69	1199.72	1.25	3600
			7	1350.78	1371.16	1.49	3600	1441.23	4.86	6.06	1	1353.90	1371.16	1.26	3600
30	3	3	8	1243.33	1345.49	7.59	3600	1350.05	0.34	6.90	1	1256.91	1345.49	6.58	3600
			9	1413.98	1434.38	1.42	3600	1551.10	7.52	8.83	1	1414.12	1434.38	1.41	3600
			0	1268.09	1331.85	4.79	3600	1367.85	2.63	6.78	1	1275.12	1329.30	4.08	3600
			1	1287.89	1287.89	0.00	456	1365.36	5.67	5.67	1	1287.89	1287.89	0.00	350
			2	1302.28	1302.28	0.00	291	1396.18	6.73	6.73	1	1302.28	1302.28	0.00	240
			3	1517.91	1517.91	0.00	109	1582.99	4.11	4.11	1	1517.91	1517.91	0.00	60
			4	1204.83	1204.83	0.00	116	1248.72	3.51	3.51	1	1204.83	1204.83	0.00	94
			5	1094.72	1174.37	6.78	3600	1251.11	6.13	12.20	1	1098.49	1169.82	6.10	3600
			6	1302.50	1302.50	0.00	1354	1380.88	5.68	5.68	1	1302.50	1302.50	0.00	1127
	1	1	7	1506.77	1506.77	0.00	706	1563.62	3.64	3.64	1	1506.77	1506.77	0.00	608
			8	1316.79	1330.50	1.03	3600	1407.03	5.44	6.04	1	1322.10	1330.50	0.63	3600
			9	1308.66	1308.66	0.00	868	1348.44	2.95	2.95	1	1308.66	1308.66	0.00	735
			0	1440.08	1558.12	7.58	3600	1651.02	5.63	11.99	1	1453.01	1552.57	6.41	3600
			1	1368.64	1523.67	10.17	3600	1604.23	5.02	13.88	1	1381.59	1511.84	8.62	3600
			2	1892.81	1892.81	0.00	2879	2030.91	6.80	6.80	0	1892.81	1892.81	0.00	2280
			3	1512.76	1537.66	1.62	3600	1660.20	7.38	8.52	1	1518.68	1536.63	1.17	3600
			4	1411.69	1612.99	12.48	3600	1608.85	-0.26	11.50	1	1423.83	1597.43	10.87	3600
			5	1489.06	1572.50	5.31	3600	1716.59	8.39	13.14	1	1491.10	1567.40	4.87	3600
	2	2	6	1826.05	1826.05	0.00	563	1901.31	3.96	3.96	0	1826.05	1826.05	0.00	459
			7	1456.76	1488.13	2.11	3600	1597.85	6.87	8.47	1	1462.59	1485.52	1.54	3600
			8	1489.68	1489.68	0.00	1211	1627.62	8.47	8.47	1	1489.68	1489.68	0.00	945
			9	1454.08	1463.82	0.67	3600	1528.52	4.23	4.73	1	1456.21	1463.82	0.52	3600
			0	1562.80	1628.16	4.01	3600	1744.24	6.66	9.92	1	1571.22	1600.73	1.84	3600
			1	-	-	-	3600	1297.15	-	11.30	1	1150.57	1282.71	10.30	3600
			2	1511.09	1648.79	8.35	3600	1654.86	0.37	8.16	1	1519.90	1622.10	6.30	3600
			3	1224.25	1341.17	8.72	3600	1443.11	7.06	14.78	1	1229.79	1334.87	7.87	3600
			4	-	-	-	3600	1562.61	-	10.62	1	1396.63	1539.26	9.27	3600
	3	3	5	-	-	-	3600	1492.59	-	11.02	1	1328.06	1479.20	10.22	3600
			6	1540.55	1558.31	1.14	3600	1640.86	5.03	5.91	1	1543.86	1558.31	0.93	3600
			7	1602.73	1602.73	0.00	2035	1707.38	6.13	6.13	1	1602.73	1602.73	0.00	1613
			8	1332.66	1397.26	4.62	3600	1533.27	8.87	12.58	1	1340.46	1372.61	2.34	3600
			9	1201.14	1382.98	13.15	3600	1496.49	7.59	18.75	1	1215.83	1365.21	10.94	3600
			0	1441.28	1448.36	0.49	3600	1595.72	9.23	9.63	2	1442.05	1448.36	0.44	3600
			1	1149.70	1423.21	19.22	3600	1408.32	-1.06	17.19	2	1166.18	1410.84	17.34	3600
			2	1435.63	1573.89	8.78	3600	1658.97	5.13	13.15	1	1440.79	1562.19	7.77	3600
			3	-	-	-	3600	1492.59	-	6.48	3	1395.88	1475.08	5.37	3600
			4	1957.16	1957.16	0.00	657	2152.12	9.06	9.06	1	1957.16	1957.16	0.00	567
	3	3	5	1658.42	1681.16	1.35	3600	1793.75	6.28	7.44	1	1660.37	1677.68	1.03	3600
			6	1607.70	1607.70	0.00	638	1722.38	6.66	6.66	1	1607.70	1607.70	0.00	558
			7	1234.90	1338.70	7.75	3600	1386.46	3.44	10.20	1	1245.03	1325.18	6.05	3600
			8	1469.13	1475.74	0.45	3600	1623.87	9.12	9.43	1	1470.80	1475.74	0.33	3600
			9	1359.42	1540.18	11.74	3600	1572.60	2.06	12.83	2	1370.88	1502.90	8.78	3600

Table 11: Exact methods on instances of [17] - tight time windows

$\mathcal{R}$	$\mathcal{T}$	TW	Variant	MILP [17]				LBBD			
				LB	UB	Gap	Time	LB	UB	Gap	Time
	4	180L	0	434.51	434.51	0.00	8	434.51	434.51	0.00	33
	4	180L	1	349.91	349.91	0.00	5	349.91	349.91	0.00	19
	4	180L	2	470.34	470.34	0.00	12	470.34	470.34	0.00	26
	4	180L	3	371.35	371.35	0.00	8	371.35	371.35	0.00	13
	4	180L	4	368.94	368.94	0.00	8	368.94	368.94	0.00	45
	4	180L	5	363.06	363.06	0.00	6	363.06	363.06	0.00	15
	4	180L	6	437.78	437.78	0.00	2	437.78	437.78	0.00	1

4	180L	7	435.27	435.27	0.00	2	435.27	435.27	0.00	4
4	180L	8	427.50	427.50	0.00	19	427.50	427.50	0.00	67
4	180L	9	458.82	458.82	0.00	4	458.82	458.82	0.00	7
4	180M	0	225.26	225.26	0.00	6	225.26	225.26	0.00	14
4	180M	1	331.20	331.20	0.00	8	331.20	331.20	0.00	15
4	180M	2	316.40	316.40	0.00	1	316.40	316.40	0.00	2
4	180M	3	337.82	337.82	0.00	5	337.82	337.82	0.00	8
4	180M	4	308.21	308.21	0.00	1	308.21	308.21	0.00	1
4	180M	5	319.61	319.61	0.00	5	319.61	319.61	0.00	6
4	180M	6	354.86	354.86	0.00	11	354.86	354.86	0.00	29
4	180M	7	309.30	309.30	0.00	14	309.30	309.30	0.00	32
4	180M	8	326.05	326.05	0.00	5	326.05	326.05	0.00	11
4	180M	9	480.44	480.44	0.00	25	480.44	480.44	0.00	60
4	180S	0	257.10	257.10	0.00	0	257.10	257.10	0.00	0
4	180S	1	222.20	222.20	0.00	0	222.20	222.20	0.00	0
4	180S	2	250.26	250.26	0.00	6	250.26	250.26	0.00	38
4	180S	3	238.00	238.00	0.00	0	238.00	238.00	0.00	0
4	180S	4	284.68	284.68	0.00	2	284.68	284.68	0.00	2
4	180S	5	290.76	290.76	0.00	10	290.76	290.76	0.00	9
4	180S	6	235.90	235.90	0.00	5	235.90	235.90	0.00	8
4	180S	7	243.25	243.25	0.00	4	243.25	243.25	0.00	4
4	180S	8	308.01	308.01	0.00	1	308.01	308.01	0.00	8
4	180S	9	283.10	283.10	0.00	4	283.10	283.10	0.00	4
4	240L	0	365.75	365.75	0.00	3	365.75	365.75	0.00	7
4	240L	1	306.65	306.65	0.00	1	306.65	306.65	0.00	7
4	240L	2	430.19	430.19	0.00	7	430.19	430.19	0.00	18
4	240L	3	343.25	343.25	0.00	6	343.25	343.25	0.00	15
4	240L	4	350.46	350.46	0.00	7	350.46	350.46	0.00	20
4	240L	5	317.20	317.20	0.00	2	317.20	317.20	0.00	5
4	240L	6	437.78	437.78	0.00	2	437.78	437.78	0.00	1
4	240L	7	435.27	435.27	0.00	3	435.27	435.27	0.00	5
4	240L	8	323.87	323.87	0.00	8	323.87	323.87	0.00	10
4	240L	9	456.30	456.30	0.00	4	456.30	456.30	0.00	12
4	240M	0	180.12	180.12	0.00	0	180.12	180.12	0.00	0
4	240M	1	309.83	309.83	0.00	4	309.83	309.83	0.00	7
4	240M	2	279.11	279.11	0.00	0	279.11	279.11	0.00	0
4	240M	3	333.37	333.37	0.00	5	333.37	333.37	0.00	9
4	240M	4	288.91	288.91	0.00	0	288.91	288.91	0.00	0
4	240M	5	298.07	298.07	0.00	1	298.07	298.07	0.00	3
4	240M	6	327.77	327.77	0.00	7	327.77	327.77	0.00	11
4	240M	7	302.88	302.88	0.00	13	302.88	302.88	0.00	28
4	240M	8	299.70	299.70	0.00	1	299.70	299.70	0.00	1
4	240M	9	377.43	377.43	0.00	5	377.43	377.43	0.00	6
4	240S	0	257.10	257.10	0.00	0	257.10	257.10	0.00	0
4	240S	1	222.20	222.20	0.00	0	222.20	222.20	0.00	0
4	240S	2	209.43	209.43	0.00	2	209.43	209.43	0.00	3
4	240S	3	238.00	238.00	0.00	0	238.00	238.00	0.00	0
4	240S	4	284.68	284.68	0.00	2	284.68	284.68	0.00	2
4	240S	5	290.18	290.18	0.00	5	290.18	290.18	0.00	7
4	240S	6	220.16	220.16	0.00	2	220.16	220.16	0.00	1
4	240S	7	243.25	243.25	0.00	3	243.25	243.25	0.00	4
4	240S	8	268.27	268.27	0.00	0	268.27	268.27	0.00	0
4	240S	9	283.10	283.10	0.00	5	283.10	283.10	0.00	4
4	300L	0	365.75	365.75	0.00	3	365.75	365.75	0.00	8
4	300L	1	306.65	306.65	0.00	2	306.65	306.65	0.00	2
4	300L	2	427.99	427.99	0.00	9	427.99	427.99	0.00	18
4	300L	3	343.25	343.25	0.00	12	343.25	343.25	0.00	15
4	300L	4	350.46	350.46	0.00	5	350.46	350.46	0.00	20
4	300L	5	317.20	317.20	0.00	5	317.20	317.20	0.00	2
4	300L	6	437.78	437.78	0.00	1	437.78	437.78	0.00	1
4	300L	7	435.27	435.27	0.00	5	435.27	435.27	0.00	4
4	300L	8	323.87	323.87	0.00	8	323.87	323.87	0.00	7
4	300L	9	456.30	456.30	0.00	6	456.30	456.30	0.00	11
4	300M	0	180.12	180.12	0.00	0	180.12	180.12	0.00	0
4	300M	1	309.83	309.83	0.00	5	309.83	309.83	0.00	5
4	300M	2	279.11	279.11	0.00	0	279.11	279.11	0.00	0
4	300M	3	333.37	333.37	0.00	6	333.37	333.37	0.00	9
4	300M	4	288.91	288.91	0.00	0	288.91	288.91	0.00	0
4	300M	5	298.07	298.07	0.00	0	298.07	298.07	0.00	3

4	300M	6	327.77	327.77	0.00	6	327.77	327.77	0.00	11
4	300M	7	302.88	302.88	0.00	14	302.88	302.88	0.00	28
4	300M	8	299.70	299.70	0.00	1	299.70	299.70	0.00	2
4	300M	9	377.43	377.43	0.00	10	377.43	377.43	0.00	7
4	300S	0	257.10	257.10	0.00	0	257.10	257.10	0.00	0
4	300S	1	222.20	222.20	0.00	0	222.20	222.20	0.00	0
4	300S	2	209.43	209.43	0.00	1	209.43	209.43	0.00	3
4	300S	3	238.00	238.00	0.00	0	238.00	238.00	0.00	0
4	300S	4	284.68	284.68	0.00	1	284.68	284.68	0.00	2
4	300S	5	290.18	290.18	0.00	6	290.18	290.18	0.00	7
4	300S	6	220.16	220.16	0.00	2	220.16	220.16	0.00	1
4	300S	7	243.25	243.25	0.00	5	243.25	243.25	0.00	4
4	300S	8	268.27	268.27	0.00	0	268.27	268.27	0.00	0
4	300S	9	283.10	283.10	0.00	6	283.10	283.10	0.00	4
5	180L	0	434.51	434.51	0.00	60	434.51	434.51	0.00	74
5	180L	1	321.45	321.45	0.00	7	321.45	321.45	0.00	14
5	180L	2	438.95	438.95	0.00	61	438.95	438.95	0.00	31
5	180L	3	371.35	371.35	0.00	44	371.35	371.35	0.00	26
5	180L	4	368.94	368.94	0.00	93	368.94	368.94	0.00	59
5	180L	5	363.06	363.06	0.00	18	363.06	363.06	0.00	19
5	180L	6	437.78	437.78	0.00	2	437.78	437.78	0.00	2
5	180L	7	435.27	435.27	0.00	5	435.27	435.27	0.00	8
5	180L	8	378.13	378.13	0.00	24	378.13	378.13	0.00	128
5	180L	9	458.82	458.82	0.00	11	458.82	458.82	0.00	17
5	180M	0	225.26	225.26	0.00	18	225.26	225.26	0.00	37
5	180M	1	331.20	331.20	0.00	16	331.20	331.20	0.00	24
5	180M	2	316.40	316.40	0.00	4	316.40	316.40	0.00	3
5	180M	3	337.82	337.82	0.00	12	337.82	337.82	0.00	16
5	180M	4	308.21	308.21	0.00	0	308.21	308.21	0.00	2
5	180M	5	319.61	319.61	0.00	6	319.61	319.61	0.00	7
5	180M	6	354.86	354.86	0.00	22	354.86	354.86	0.00	54
5	180M	7	292.28	292.28	0.00	26	292.28	292.28	0.00	77
5	180M	8	326.05	326.05	0.00	8	326.05	326.05	0.00	20
5	180M	9	473.58	473.58	0.00	143	473.58	473.58	0.00	129
5	180S	0	257.10	257.10	0.00	0	257.10	257.10	0.00	0
5	180S	1	222.20	222.20	0.00	0	222.20	222.20	0.00	0
5	180S	2	247.03	247.03	0.00	11	247.03	247.03	0.00	60
5	180S	3	238.00	238.00	0.00	0	238.00	238.00	0.00	0
5	180S	4	284.68	284.68	0.00	3	284.68	284.68	0.00	2
5	180S	5	290.76	290.76	0.00	12	290.76	290.76	0.00	18
5	180S	6	235.90	235.90	0.00	11	235.90	235.90	0.00	14
5	180S	7	243.25	243.25	0.00	12	243.25	243.25	0.00	11
5	180S	8	308.01	308.01	0.00	1	308.01	308.01	0.00	13
5	180S	9	283.10	283.10	0.00	6	283.10	283.10	0.00	11
5	240L	0	365.75	365.75	0.00	6	365.75	365.75	0.00	13
5	240L	1	306.65	306.65	0.00	4	306.65	306.65	0.00	4
5	240L	2	404.54	404.54	0.00	13	404.54	404.54	0.00	27
5	240L	3	343.25	343.25	0.00	38	343.25	343.25	0.00	15
5	240L	4	350.46	350.46	0.00	17	350.46	350.46	0.00	27
5	240L	5	317.20	317.20	0.00	15	317.20	317.20	0.00	5
5	240L	6	437.78	437.78	0.00	4	437.78	437.78	0.00	2
5	240L	7	435.27	435.27	0.00	12	435.27	435.27	0.00	9
5	240L	8	323.87	323.87	0.00	9	323.87	323.87	0.00	12
5	240L	9	438.87	438.87	0.00	20	438.87	438.87	0.00	12
5	240M	0	180.12	180.12	0.00	1	180.12	180.12	0.00	0
5	240M	1	309.83	309.83	0.00	9	309.83	309.83	0.00	10
5	240M	2	279.11	279.11	0.00	1	279.11	279.11	0.00	0
5	240M	3	333.37	333.37	0.00	9	333.37	333.37	0.00	14
5	240M	4	288.91	288.91	0.00	0	288.91	288.91	0.00	1
5	240M	5	298.07	298.07	0.00	1	298.07	298.07	0.00	3
5	240M	6	327.77	327.77	0.00	15	327.77	327.77	0.00	14
5	240M	7	292.28	292.28	0.00	34	292.28	292.28	0.00	78
5	240M	8	299.70	299.70	0.00	2	299.70	299.70	0.00	0
5	240M	9	377.43	377.43	0.00	14	377.43	377.43	0.00	12
5	240S	0	257.10	257.10	0.00	0	257.10	257.10	0.00	0
5	240S	1	222.20	222.20	0.00	0	222.20	222.20	0.00	0
5	240S	2	209.43	209.43	0.00	1	209.43	209.43	0.00	4
5	240S	3	238.00	238.00	0.00	0	238.00	238.00	0.00	0

5	240S	4	284.68	284.68	0.00	2	284.68	284.68	0.00	2
5	240S	5	290.18	290.18	0.00	14	290.18	290.18	0.00	16
5	240S	6	220.16	220.16	0.00	6	220.16	220.16	0.00	4
5	240S	7	243.25	243.25	0.00	6	243.25	243.25	0.00	12
5	240S	8	268.27	268.27	0.00	0	268.27	268.27	0.00	1
5	240S	9	283.10	283.10	0.00	12	283.10	283.10	0.00	11
5	300L	0	365.75	365.75	0.00	8	365.75	365.75	0.00	13
5	300L	1	306.65	306.65	0.00	4	306.65	306.65	0.00	3
5	300L	2	402.93	402.93	0.00	14	402.93	402.93	0.00	19
5	300L	3	343.25	343.25	0.00	37	343.25	343.25	0.00	17
5	300L	4	350.46	350.46	0.00	11	350.46	350.46	0.00	28
5	300L	5	317.20	317.20	0.00	8	317.20	317.20	0.00	4
5	300L	6	437.78	437.78	0.00	6	437.78	437.78	0.00	2
5	300L	7	435.27	435.27	0.00	11	435.27	435.27	0.00	12
5	300L	8	323.87	323.87	0.00	16	323.87	323.87	0.00	14
5	300L	9	438.87	438.87	0.00	13	438.87	438.87	0.00	8
5	300M	0	180.12	180.12	0.00	5	180.12	180.12	0.00	0
5	300M	1	309.83	309.83	0.00	9	309.83	309.83	0.00	7
5	300M	2	279.11	279.11	0.00	1	279.11	279.11	0.00	0
5	300M	3	333.37	333.37	0.00	10	333.37	333.37	0.00	17
5	300M	4	288.91	288.91	0.00	0	288.91	288.91	0.00	0
5	300M	5	298.07	298.07	0.00	1	298.07	298.07	0.00	4
5	300M	6	327.77	327.77	0.00	15	327.77	327.77	0.00	13
5	300M	7	292.28	292.28	0.00	40	292.28	292.28	0.00	72
5	300M	8	299.70	299.70	0.00	2	299.70	299.70	0.00	1
5	300M	9	377.43	377.43	0.00	11	377.43	377.43	0.00	10
5	300S	0	257.10	257.10	0.00	1	257.10	257.10	0.00	0
5	300S	1	222.20	222.20	0.00	0	222.20	222.20	0.00	0
5	300S	2	209.43	209.43	0.00	2	209.43	209.43	0.00	4
5	300S	3	238.00	238.00	0.00	0	238.00	238.00	0.00	0
5	300S	4	284.68	284.68	0.00	2	284.68	284.68	0.00	2
5	300S	5	290.18	290.18	0.00	10	290.18	290.18	0.00	14
5	300S	6	220.16	220.16	0.00	8	220.16	220.16	0.00	4
5	300S	7	243.25	243.25	0.00	7	243.25	243.25	0.00	13
5	300S	8	268.27	268.27	0.00	0	268.27	268.27	0.00	1
5	300S	9	283.10	283.10	0.00	6	283.10	283.10	0.00	12
4	180L	0	511.75	511.75	0.00	102	511.75	511.75	0.00	226
4	180L	1	413.59	413.59	0.00	136	413.59	413.59	0.00	155
4	180L	2	592.19	592.19	0.00	504	592.19	592.19	0.00	181
4	180L	3	388.82	388.82	0.00	60	388.82	388.82	0.00	26
4	180L	4	450.52	450.52	0.00	343	450.52	450.52	0.00	565
4	180L	5	419.65	419.65	0.00	132	419.65	419.65	0.00	360
4	180L	6	584.09	584.09	0.00	36	584.09	584.09	0.00	27
4	180L	7	453.87	453.87	0.00	11	453.87	453.87	0.00	10
4	180L	8	532.22	532.22	0.00	319	532.22	532.22	0.00	1541
4	180L	9	566.48	566.48	0.00	31	566.48	566.48	0.00	38
4	180M	0	355.39	355.39	0.00	34	355.39	355.39	0.00	33
4	180M	1	498.85	521.75	4.39	3600	521.75	521.75	0.00	1988
4	180M	2	328.56	328.56	0.00	31	328.56	328.56	0.00	5
4	180M	3	356.53	356.53	0.00	20	356.53	356.53	0.00	43
4	180M	4	379.22	379.22	0.00	14	379.22	379.22	0.00	3
4	180M	5	395.78	395.78	0.00	17	395.78	395.78	0.00	110
4	180M	6	386.59	386.59	0.00	370	386.59	386.59	0.00	467
4	180M	7	424.59	424.59	0.00	597	424.59	424.59	0.00	1163
4	180M	8	505.92	505.92	0.00	604	505.92	505.92	0.00	322
4	180M	9	615.64	615.64	0.00	1542	615.64	615.64	0.00	1052
4	180S	0	377.71	377.71	0.00	48	377.71	377.71	0.00	28
4	180S	1	338.30	338.30	0.00	8	338.30	338.30	0.00	1
4	180S	2	300.67	300.67	0.00	438	300.67	300.67	0.00	206
4	180S	3	320.17	320.17	0.00	0	320.17	320.17	0.00	0
4	180S	4	350.97	350.97	0.00	16	350.97	350.97	0.00	3
4	180S	5	337.04	337.04	0.00	298	337.04	337.04	0.00	203
4	180S	6	301.89	301.89	0.00	245	301.89	301.89	0.00	19
4	180S	7	266.32	266.32	0.00	84	266.32	266.32	0.00	2
4	180S	8	381.79	381.79	0.00	13	381.79	381.79	0.00	18
4	180S	9	370.44	370.44	0.00	34	370.44	370.44	0.00	15
4	240L	0	405.41	405.41	0.00	24	405.41	405.41	0.00	4
4	240L	1	356.04	356.04	0.00	22	356.04	356.04	0.00	21

4	240L	2	488.66	488.66	0.00	78	488.66	488.66	0.00	106
4	240L	3	388.82	388.82	0.00	130	388.82	388.82	0.00	33
4	240L	4	403.40	403.40	0.00	61	403.40	403.40	0.00	107
4	240L	5	419.65	419.65	0.00	268	419.65	419.65	0.00	255
4	240L	6	540.06	540.06	0.00	25	540.06	540.06	0.00	11
4	240L	7	453.87	453.87	0.00	16	453.87	453.87	0.00	9
4	240L	8	452.90	452.90	0.00	181	452.90	452.90	0.00	101
4	240L	9	549.05	549.05	0.00	46	549.05	549.05	0.00	29
4	240M	0	310.25	310.25	0.00	4	310.25	310.25	0.00	2
4	240M	1	420.40	420.40	0.00	223	420.40	420.40	0.00	178
4	240M	2	297.34	297.34	0.00	7	297.34	297.34	0.00	0
4	240M	3	356.53	356.53	0.00	27	356.53	356.53	0.00	18
4	240M	4	379.22	379.22	0.00	6	379.22	379.22	0.00	4
4	240M	5	374.24	374.24	0.00	11	374.24	374.24	0.00	47
4	240M	6	338.14	338.14	0.00	29	338.14	338.14	0.00	15
4	240M	7	384.68	384.68	0.00	264	384.68	384.68	0.00	188
4	240M	8	419.09	419.09	0.00	24	419.09	419.09	0.00	8
4	240M	9	464.95	464.95	0.00	39	464.95	464.95	0.00	35
4	240S	0	377.71	377.71	0.00	29	377.71	377.71	0.00	32
4	240S	1	338.30	338.30	0.00	2	338.30	338.30	0.00	0
4	240S	2	282.12	282.12	0.00	214	282.12	282.12	0.00	80
4	240S	3	320.17	320.17	0.00	0	320.17	320.17	0.00	0
4	240S	4	350.97	350.97	0.00	18	350.97	350.97	0.00	3
4	240S	5	290.90	290.90	0.00	35	290.90	290.90	0.00	5
4	240S	6	301.00	301.00	0.00	166	301.00	301.00	0.00	19
4	240S	7	266.32	266.32	0.00	25	266.32	266.32	0.00	2
4	240S	8	342.04	342.04	0.00	1	342.04	342.04	0.00	1
4	240S	9	354.54	354.54	0.00	21	354.54	354.54	0.00	2
4	300L	0	405.41	405.41	0.00	20	405.41	405.41	0.00	6
4	300L	1	356.04	356.04	0.00	19	356.04	356.04	0.00	27
4	300L	2	488.66	488.66	0.00	138	488.66	488.66	0.00	101
4	300L	3	388.82	388.82	0.00	223	388.82	388.82	0.00	35
4	300L	4	403.40	403.40	0.00	41	403.40	403.40	0.00	106
4	300L	5	419.65	419.65	0.00	185	419.65	419.65	0.00	250
4	300L	6	540.06	540.06	0.00	29	540.06	540.06	0.00	11
4	300L	7	453.87	453.87	0.00	22	453.87	453.87	0.00	12
4	300L	8	441.78	441.78	0.00	128	441.78	441.78	0.00	71
4	300L	9	549.05	549.05	0.00	70	549.05	549.05	0.00	29
4	300M	0	310.25	310.25	0.00	7	310.25	310.25	0.00	1
4	300M	1	420.40	420.40	0.00	190	420.40	420.40	0.00	174
4	300M	2	297.34	297.34	0.00	5	297.34	297.34	0.00	1
4	300M	3	356.53	356.53	0.00	32	356.53	356.53	0.00	13
4	300M	4	379.22	379.22	0.00	3	379.22	379.22	0.00	2
4	300M	5	374.24	374.24	0.00	10	374.24	374.24	0.00	51
4	300M	6	338.14	338.14	0.00	45	338.14	338.14	0.00	13
4	300M	7	384.68	384.68	0.00	271	384.68	384.68	0.00	183
4	300M	8	419.09	419.09	0.00	24	419.09	419.09	0.00	6
4	300M	9	464.95	464.95	0.00	84	464.95	464.95	0.00	42
4	300S	0	377.71	377.71	0.00	19	377.71	377.71	0.00	32
4	300S	1	338.30	338.30	0.00	1	338.30	338.30	0.00	1
4	300S	2	282.12	282.12	0.00	248	282.12	282.12	0.00	69
4	300S	3	320.17	320.17	0.00	0	320.17	320.17	0.00	0
4	300S	4	350.97	350.97	0.00	25	350.97	350.97	0.00	3
4	300S	5	290.90	290.90	0.00	21	290.90	290.90	0.00	5
4	300S	6	301.00	301.00	0.00	71	301.00	301.00	0.00	18
4	300S	7	266.32	266.32	0.00	28	266.32	266.32	0.00	2
4	300S	8	342.04	342.04	0.00	1	342.04	342.04	0.00	1
4	300S	9	354.54	354.54	0.00	26	354.54	354.54	0.00	3
4	180L	0	602.86	602.86	0.00	638	602.86	602.86	0.00	848
4	180L	1	-	-	-	3600	-	-	-	3600
4	180L	2	588.60	608.56	3.28	3600	-	-	-	3600
4	180L	3	456.39	524.95	13.06	3600	524.95	524.95	0.00	265
4	180L	4	-	-	-	3600	-	-	-	655
4	180L	5	474.07	512.34	7.47	3600	512.34	512.34	0.00	2371
4	180L	6	-	-	-	3600	-	-	-	3600
4	180L	7	616.24	616.24	0.00	145	616.24	616.24	0.00	27
4	180L	8	-	-	-	3600	-	620.89	-	3600
4	180L	9	-	-	-	3600	-	-	-	3600

4	180M	0	478.30	478.30	0.00	3517	478.30	478.30	0.00	594
4	180M	1	-	-	-	3600	-	-	-	3600
4	180M	2	425.09	425.09	0.00	433	425.09	425.09	0.00	15
4	180M	3	446.85	446.85	0.00	633	446.85	446.85	0.00	117
4	180M	4	-	-	-	3600	-	-	-	3600
4	180M	5	444.72	500.19	11.09	3600	440.34	500.19	11.97	3600
4	180M	6	400.65	400.65	0.00	2001	400.65	400.65	0.00	859
4	180M	7	-	-	-	3600	-	515.17	-	3600
4	180M	8	553.26	553.26	0.00	912	553.26	553.26	0.00	672
4	180M	9	-	-	-	3600	-	-	-	3600
4	180S	0	425.56	425.56	0.00	24	425.56	425.56	0.00	2
4	180S	1	382.17	382.17	0.00	47	382.17	382.17	0.00	5
4	180S	2	399.87	399.87	0.00	3428	399.87	399.87	0.00	541
4	180S	3	398.02	398.02	0.00	68	398.02	398.02	0.00	15
4	180S	4	456.32	456.32	0.00	104	456.32	456.32	0.00	8
4	180S	5	396.50	396.50	0.00	754	396.50	396.50	0.00	249
4	180S	6	369.66	369.66	0.00	1215	369.66	369.66	0.00	241
4	180S	7	363.27	363.27	0.00	910	363.27	363.27	0.00	63
4	180S	8	429.12	429.12	0.00	126	429.12	429.12	0.00	91
4	180S	9	428.05	428.05	0.00	80	428.05	428.05	0.00	10
4	240L	0	509.57	509.57	0.00	354	509.57	509.57	0.00	16
4	240L	1	509.32	509.32	0.00	1296	509.32	509.32	0.00	202
4	240L	2	569.04	569.04	0.00	1778	569.04	569.04	0.00	491
4	240L	3	435.79	500.62	12.95	3600	500.62	500.62	0.00	60
4	240L	4	456.37	456.37	0.00	1422	456.37	456.37	0.00	120
4	240L	5	447.00	447.00	0.00	1173	447.00	447.00	0.00	120
4	240L	6	592.08	592.08	0.00	479	592.08	592.08	0.00	58
4	240L	7	616.24	616.24	0.00	402	616.24	616.24	0.00	33
4	240L	8	466.87	466.87	0.00	867	466.87	466.87	0.00	499
4	240L	9	615.50	615.50	0.00	1280	615.50	615.50	0.00	526
4	240M	0	446.21	446.21	0.00	840	446.21	446.21	0.00	125
4	240M	1	512.57	512.57	0.00	2767	512.57	512.57	0.00	998
4	240M	2	393.88	393.88	0.00	40	393.88	393.88	0.00	1
4	240M	3	446.85	446.85	0.00	980	446.85	446.85	0.00	263
4	240M	4	419.94	419.94	0.00	79	419.94	419.94	0.00	7
4	240M	5	418.26	418.26	0.00	425	418.26	418.26	0.00	131
4	240M	6	376.74	376.74	0.00	700	376.74	376.74	0.00	46
4	240M	7	431.42	470.57	8.32	3600	470.57	470.57	0.00	838
4	240M	8	476.13	476.13	0.00	376	476.13	476.13	0.00	261
4	240M	9	509.93	559.20	8.81	3600	559.20	559.20	0.00	1499
4	240S	0	425.56	425.56	0.00	32	425.56	425.56	0.00	2
4	240S	1	382.17	382.17	0.00	37	382.17	382.17	0.00	2
4	240S	2	381.32	381.32	0.00	1371	381.32	381.32	0.00	206
4	240S	3	398.02	398.02	0.00	43	398.02	398.02	0.00	17
4	240S	4	450.46	450.46	0.00	191	450.46	450.46	0.00	10
4	240S	5	350.37	350.37	0.00	89	350.37	350.37	0.00	5
4	240S	6	330.20	330.20	0.00	229	330.20	330.20	0.00	16
4	240S	7	363.27	363.27	0.00	632	363.27	363.27	0.00	83
4	240S	8	389.38	389.38	0.00	21	389.38	389.38	0.00	5
4	240S	9	428.05	428.05	0.00	188	428.05	428.05	0.00	9
4	300L	0	509.57	509.57	0.00	288	509.57	509.57	0.00	12
4	300L	1	509.32	509.32	0.00	1392	509.32	509.32	0.00	170
4	300L	2	569.04	569.04	0.00	1397	569.04	569.04	0.00	513
4	300L	3	429.52	512.68	16.22	3600	500.62	500.62	0.00	72
4	300L	4	456.37	456.37	0.00	1813	456.37	456.37	0.00	110
4	300L	5	447.00	447.00	0.00	1438	447.00	447.00	0.00	144
4	300L	6	592.08	592.08	0.00	763	592.08	592.08	0.00	62
4	300L	7	616.24	616.24	0.00	453	616.24	616.24	0.00	30
4	300L	8	451.36	451.36	0.00	804	451.36	451.36	0.00	249
4	300L	9	615.50	615.50	0.00	1579	615.50	615.50	0.00	365
4	300M	0	446.21	446.21	0.00	1580	446.21	446.21	0.00	100
4	300M	1	491.04	491.04	0.00	886	491.04	491.04	0.00	286
4	300M	2	393.88	393.88	0.00	96	393.88	393.88	0.00	3
4	300M	3	446.85	446.85	0.00	1509	446.85	446.85	0.00	254
4	300M	4	419.94	419.94	0.00	56	419.94	419.94	0.00	10
4	300M	5	418.26	418.26	0.00	109	418.26	418.26	0.00	127
4	300M	6	376.74	376.74	0.00	1054	376.74	376.74	0.00	46
4	300M	7	470.57	470.57	0.00	3333	470.57	470.57	0.00	780
4	300M	8	476.13	476.13	0.00	298	476.13	476.13	0.00	183

4	300M	9	502.16	559.20	10.20	3600	559.20	559.20	0.00	1365
4	300S	0	425.56	425.56	0.00	54	425.56	425.56	0.00	2
4	300S	1	382.17	382.17	0.00	28	382.17	382.17	0.00	2
4	300S	2	381.32	381.32	0.00	1286	381.32	381.32	0.00	190
4	300S	3	398.02	398.02	0.00	125	398.02	398.02	0.00	15
4	300S	4	450.46	450.46	0.00	283	450.46	450.46	0.00	8
4	300S	5	350.37	350.37	0.00	205	350.37	350.37	0.00	5
4	300S	6	330.20	330.20	0.00	165	330.20	330.20	0.00	14
4	300S	7	360.99	360.99	0.00	884	360.99	360.99	0.00	67
4	300S	8	389.38	389.38	0.00	12	389.38	389.38	0.00	5
4	300S	9	428.05	428.05	0.00	178	428.05	428.05	0.00	10